From Source Code to Sampling Correction: Predicting Memory-Footprint Models for Unseen Programs

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

MemPrint estimates how many bytes a program touches from a sparse sample of its memory references: a Pin tool keeps one reference in k, and a regression model, trained on small inputs of the same program that were traced completely, predicts the factor α by which the sampled footprint must be scaled up. The model belongs to one program, so a program that has never been traced has no model. This report asks how a model can be obtained for such an unseen program, and how the memory behaviour of two programs can be compared, when all that is known about the new program is its source code. We show that, under Bernoulli sampling, α at every sampling interval is a closed-form function of the program's *access-count spectrum*: how many bytes receive how many references. We obtain that spectrum without running the program on its data, with a vectorised abstract interpreter that executes the program's abstract syntax tree from `main()` and charges every memory reference to its address, and we add a small, shared spectrum for the runtime (loader, C library, allocator), fitted by non-negative least squares on other programs. On the 27 PolyBench kernels, with each kernel held out in turn, this predicts the footprint within a median of 0.09% and 0.27% (largest and middle input held out) and α within a median of 1.19% and 5.50%, against 18.04% and 10.85% for the kernel's own model trained on its traces and 8.89% and 6.05% for the best borrowed model chosen after the fact. A memory-behaviour descriptor z derived from the spectrum ranks which known model transfers best (median Spearman ρ of 0.83–0.85), as well as the descriptor measured from traces and far better than counts of syntax-tree nodes ($\rho = 0.27$). For irregular programs, whose memory behaviour depends on their input, we extend the approach in three steps on the GAP graph benchmarks: a hand-written skeleton that replays the program on an input drawn from the distribution of its generator (blind tests on Kronecker graphs, new kernels and OpenMP threads pass with 0.6–11% error), an interpreter that runs irregular C code, and finally a C++ interpreter that runs GAP's own source for all six kernels on both graph families at scales 10–18, predicting the footprint within 3% and α within about a point of the hand-written skeleton for pr, bfs and tc, within two points for cc and sssp, and better than it on average for bc. A blind test on two programs never used in development, with predictions committed before they were traced, passes clearly on HPCCG (footprint within 2.1%, α within 0.5–4.4%, against 11–12% for its own model) and partly on LULESH (footprint within 4.2%, α within 5% at small and middle meshes but 28–43% at the largest). A second blind test on three NAS Parallel Benchmarks predicts IS and CG almost exactly (footprint within 0.33%, α within 1.8–5.3%) and fails on MG (footprint 7–9% short, α off by more than 200%). Symbolic counting of affine innermost loops gives identical spectra in a fraction of the time, and the same interpreter predicts the footprint over time, with the peak within 3.5% on PolyBench. We measured the cost of every method afresh: on affine code the static footprint costs 0.5% of a full trace and a twentieth of MemPrint's own sampled run while being more accurate than its model; on irregular code the interpreter costs 0.3–2.2 full traces, and on programs that are long chains of dependent steps (IS, CG) up to 214, where tracing is cheaper. It needs traced programs with the same runtime

for its baseline, and programs it cannot run still need sampling. We describe every component in enough detail to re-implement it, and we report the limitations, the post-hoc corrections and the parts of the evaluation that are not blind.

Contents

1 Introduction **6**

2 Background and terminology **7**
2.1 References, addresses and the footprint . . . 7
2.2 The splitter and its bins . . . 7
2.3 The correction factor α and the per-program model . . . 8
2.4 Inputs, splits and error . . . 8
2.5 Workloads . . . 8

3 The access-count spectrum decides α **10**
3.1 Moments of a bin . . . 10
3.2 Assumptions . . . 10
3.3 A descriptor of memory behaviour . . . 11

4 Computing the spectrum from source code **12**
4.1 Overview . . . 12
4.2 Values and memory . . . 12
4.3 Batches: running a loop nest all at once . . . 12
4.4 Symbolic counting of affine innermost loops . . . 13
4.5 What is charged (`-O0`) . . . 14
4.6 Uncertainty and coverage . . . 14
4.7 Irregular C: memory values, random numbers, the heap . . . 14
4.8 Implementation constants . . . 14

5 The runtime baseline **16**

6 Using the spectrum for an unseen program **17**
6.1 The setting: what is known and what is asked . . . 17
6.2 Family A: predicted from the source . . . 17
6.3 Family B: borrowed from known programs . . . 19
6.4 Family C and the references . . . 20
6.5 The access-idiom gate . . . 21

7 Evaluation on PolyBench **23**
7.1 Protocol . . . 23
7.2 Accuracy . . . 23
7.3 The moments match Pin . . . 23
7.4 The footprint over time from source . . . 25
7.5 Dependence on the runtime baseline . . . 25
7.6 Similarity: does $\hat{z}$ rank transferable models? . . . 28
7.7 Cost of each method . . . 30
7.8 The gate on irregular programs . . . 31

8 Irregular code I: a skeleton and the input's distribution **33**
8.1 The input distribution . . . 33
8.2 The skeleton . . . 33
8.3 The glibc heap model . . . 33
8.4 Calibration and blind tests . . . 34
8.5 Results . . . 34
8.6 Irregular C programs run by the interpreter . . . 34

9 Irregular code II: interpreting GAP's C++ source **38**
9.1 Object model . . . 38
9.2 Values . . . 38
9.3 Counting as `-O3` does . . . 39
9.4 The standard library . . . 39
9.5 Control flow . . . 39
9.6 Bugs found by comparing against the hand skeleton . . . 40
9.7 Results . . . 40
9.8 Cost . . . 43

10 A blind test on programs never seen in development **46**

11 A second blind test: NPB **48**

12 The cost of every method **50**

13 Discussion **53**

14 Related work **54**

15 Conclusion **55**

A Per-kernel leave-one-out results **58**

B Commands **58**

List of Figures

1 The prediction pipeline . . . 7
2 Inclusion probability and the α of idealised spectra . . . 10
3 Symbolic counting . . . 13
4 Runtime baselines . . . 16
5 What each method needs . . . 18
6 The three families of methods . . . 18
7 Worked example of the static route . . . 19
8 Borrowing in practice . . . 21
9 Leave-one-out errors by method . . . 24
10 Static α against the own model . . . 24
11 Static footprint error per kernel . . . 24
12 Predicted and measured α curves on PolyBench . . . 26
13 Predicted against measured bin moments . . . 26

14 Static α error per kernel and size . . . 27
15 Error of the predicted spread . . . 27
16 Static spectra of PolyBench . . . 27
17 The footprint over time from source . . . 28
18 Validity of the descriptor . . . 29
19 Predicted and measured descriptor curves . . . 29
20 Cost and reference share of the C interpreter . . . 31
21 The access-idiom gate . . . 32
22 Borrowing PolyBench models for miniVite . . . 32
23 Degree distributions of GAP's generators . . . 35
24 Skeleton footprint error . . . 35
25 GAP: skeleton against borrowed models . . . 35
26 Skeleton error per scale . . . 37
27 Interpreter on irregular C programs . . . 37
28 C++ interpreter against the hand skeleton, per scale . . . 42
29 Predicted and measured α curves on GAP . . . 43
30 C++ interpreter footprint error . . . 43
31 Hand and interpreter spectra . . . 44
32 Per-block comparison . . . 44
33 Cost of the C++ interpreter . . . 45
34 Blind test . . . 47

List of Tables

1 Glossary of the terms used in this report. . . . 9
2 Constants of the interpreters and skeletons. . . . 15
3 Runtime baseline coefficients . . . 16
4 The methods at a glance . . . 21
5 Leave-one-out accuracy on PolyBench . . . 23
6 Dependence on the runtime baseline . . . 27
7 Spearman ρ between descriptor distance and transfer error, over the 27 held-out kernels. . . . 28
8 Cost of each method . . . 30
9 Access idioms of PolyBench and of irregular programs. . . . 31
10 Measured references per element of `std::sort` . . . 33
11 Skeleton results on GAP . . . 36
12 Interpreter on irregular C programs . . . 37
13 Interpreter defects found on GAP and their fixes. . . . 41
14 C++ interpreter against the hand skeleton . . . 42
15 pr and bfs per scale . . . 43
16 C++ interpreter seconds per kernel and scale as frozen for the second blind test (u: uniform, K: Kronecker; recorded with 4–16 runs at a time, so up to about twice the time of a run alone, Table 22). . . . 45
17 Blind test per config . . . 46
18 Blind test against the own model . . . 47
19 Second blind test: NPB . . . 48
20 Cost of every method . . . 50

21 Cost of the analysis steps 51
22 Re-measured against earlier times 52
23 Per-kernel leave-one-out results 59

1 Introduction

The memory footprint of a program, the number of distinct bytes it touches during a run, decides how many jobs fit on a node, how large a cache or memory tier must be, and whether a program fits in device memory at all. The direct way to measure it is to record every memory reference with a dynamic binary instrumentation tool such as Pin [20]; this costs two to three orders of magnitude in run time. MemPrint [25] reduces that cost by recording only a random subset of the references: each reference is kept with probability $1/k$. The bytes touched by the kept references, the *sampled footprint* m, underestimates the true footprint, because bytes that are referenced only a few times are often missed. MemPrint learns the correction: for each program it fits a regression model that predicts the ratio $\alpha = \text{truth}/m$ from m, k and the spread of m across independent subsamples, using small inputs of the same program that were traced completely.

That model is specific to one program. Its training needs complete traces of that program at several input sizes; a program that was never traced has no model. A natural idea is to reuse the model of a similar program, but this raises two questions that this report answers:

1. **Similarity.** Given models for programs A, B and D and a new program C whose memory behaviour "falls between" B and D, how can memory-behaviour similarity be quantified so that it predicts which model transfers?
2. **From code.** If nothing is known about C except its source code, which static features of the code determine where C falls, and how does the code translate into the memory-behaviour descriptor $z(C)$?

Our answer to both is the same object, the *access-count spectrum* of the program (Section 3). Under Bernoulli sampling, the spectrum determines the mean and variance of the sampled footprint at every interval k in closed form. It therefore determines α directly (no model needs to be borrowed at all), it defines a descriptor z in which distances between programs predict transfer error, and it can be computed from source code by abstract interpretation [7].

Contributions.

- A derivation showing that α, the bins' spread and their unique-address counts follow from the spectrum (Section 3), and a descriptor z of memory behaviour with two parts, *reuse* and *spread* (Section 3.3).
- A vectorised abstract interpreter for C that computes the spectrum from source (Section 4), a shared runtime baseline fitted by non-negative least squares (Section 5), and an access-idiom gate that recognises code outside the interpreter's reach (Section 6.5).
- A leave-one-workload-out evaluation on 27 PolyBench kernels comparing eleven ways to predict an unseen kernel (Section 7).
- An extension to irregular, input-dependent code on the GAP benchmark suite [2]: a skeleton replay on inputs sampled from the generator's distribution, with blind tests (Section 8), an interpreter for irregular C (Section 4.7) and a C++ interpreter that runs GAP's unmodified source (Section 9).

Figure 1 shows the whole pipeline. Section 2 defines the terms used throughout; Table 1 collects them.

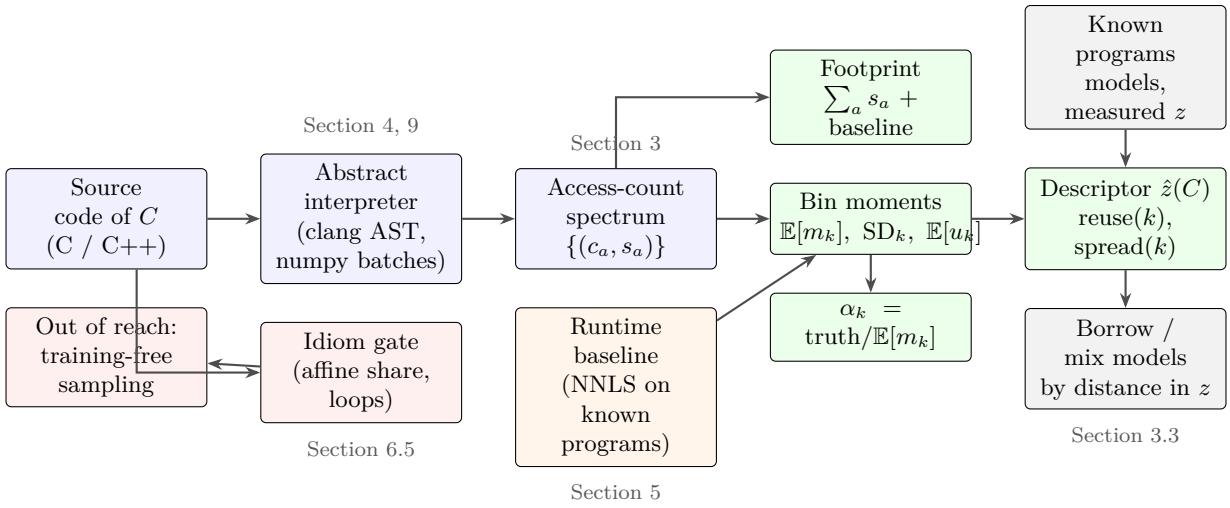


Figure 1: The prediction pipeline for an unseen program C. Blue: computed from C's source alone. Orange: fitted on other programs. Green: quantities that follow in closed form. Gray: what borrowing a known model needs. Red: the check that sends code the analysis cannot handle to a training-free method.

2 Background and terminology

2.1 References, addresses and the footprint

A *memory reference* (or *access*) is one load or store executed by the program; it covers a byte range [addr, addr+size). An *address* in this report is a 4-byte *unit*: the interpreter and the skeletons count references per aligned 4-byte unit, and a reference counts once for every unit it covers, whatever its width. The *footprint* of a run is the number of distinct bytes covered by at least one reference (*bytes touched*). This is MemPrint's `-footprint bytes` mode. The original definition, `-footprint starts`, adds up the largest access size seen at each start address and counts overlapping accesses more than once; we use bytes touched throughout, and all traces in this report were recorded with it.

2.2 The splitter and its bins

MemPrint's Pin tool in *splitter* mode records every reference of a run once and produces, from that single run, both the exact footprint and many subsamples. For each *sampling interval* $k \in \mathcal{K} = \{100, 250, 500, 750, 1000, 2500, 5000, 7500, 10^4, 2.5 \cdot 10^4, 5 \cdot 10^4, 7.5 \cdot 10^4, 10^5\}$ it keeps $B = 20$ *bins*. Each bin keeps every reference independently with probability $1/k$ (a Bernoulli subsample, drawn with an independent random stream per bin). For a bin the tool reports:

- `MemUsageObs`, the *bin footprint* m: the bytes touched by the bin's references;
- `UniqueAddresses` u: the number of distinct addresses among them;
- `CountObs`: the number of references the bin kept.

The interval $k = 1$ is the complete trace: its `MemUsageObs` is the true footprint, *truth*. The tables of all bins of all inputs of a program are the program's *traces* (`allData` files).

2.3 The correction factor α and the per-program model

For one bin at interval k, $\alpha = \text{truth}/m$. MemPrint predicts it with a log-linear model with interactions:

$$\log \alpha = b_0 + b_1\ell_\sigma + b_2\ell_k + b_3\ell_m + b_4\ell_m\ell_\sigma + b_5\ell_k\ell_\sigma + b_6\ell_m\ell_k + b_7\ell_m\ell_\sigma\ell_k, \tag{1}$$

with $\ell_\sigma = \log \mathrm{SD}_k$ (the standard deviation of m across the B bins of that interval), $\ell_k = \log k$ and $\ell_m = \log m$. Rows with $k \in \{1, 10\}$ and intervals with an empty bin are dropped. Training uses the other inputs of the same program. We call this the program's *own model*.

2.4 Inputs, splits and error

Each program has several *configs* (input sizes), ordered by true footprint. Two *splits* are evaluated: *EXTRA* holds out the largest config (extrapolation) and *INTER* the middle one (interpolation). Accuracy is the *MAPE*, the mean absolute percentage error of predicted α against measured α over all bins of all 13 intervals at the held-out config; for the footprint it is the absolute percentage error of the predicted truth.

2.5 Workloads

- **PolyBench/C 4.2.1** [28]: 27 kernels (all but deriche, durbin and jacobi-1d, as in the original MemPrint study), each at seven sizes: MINI, MINI2, MINI3, SMALL, SMALL2, SMALL3, MEDIUM. Built `-O0`. Affine loop nests over arrays.
- **GAP** [2] (commit `b5e3e19`), built with its default `-O3`: pr (PageRank, 20 iterations, `-t 0`), prc (PageRank to convergence, `-t 1e-4`), bfs (direction-optimising BFS [1]), cc (Afforest [32]), sssp (Δ-stepping [23]), tc (triangle counting) and bc (Brandes [5]), on synthetic graphs generated inside the program: *uniform* (`-u` s) or *Kronecker* (`-g` s, R-MAT [6, 18]) with 2^s vertices and average degree 16, at *scales* $s = 10, \ldots, 18$. pr and bfs were also run with 4 OpenMP threads.
- **miniVite** [12] (graph community detection, MPI), as a transfer check.
- Two small C programs written for this work, `csr_pr.c` and `csr_bfs.c`.

Table 1: Glossary of the terms used in this report.

Term	Meaning
reference, access	one executed load or store, covering a byte range
address, unit	an aligned 4-byte unit; references are counted per unit they cover
footprint, truth	bytes touched by the whole run (`-footprint bytes`)
sampling interval k	each reference is kept with probability $1/k$
bin	one independent 1-in-k subsample of a run; 20 per interval
bin footprint m	bytes touched by a bin's references (`MemUsageObs`)
SD_k, spread	standard deviation of m across the bins of interval k
α	truth/m, the factor that scales a sampled footprint up
own model	(1) fitted on the program's other inputs
config, scale	an input size; for GAP the scale s means 2^s vertices
EXTRA, INTER	largest / middle config held out
MAPE	mean absolute percentage error over all bins at the held-out config
access-count spectrum	the multiset $\{(c_a, s_a)\}$: address a of s_a bytes receives c_a references
inclusion probability	$p(c,k) = 1-(1-1/k)^c$, the chance a bin holds an address with c references
runtime baseline	spectrum of what the loader, C library and allocator touch, outside the program text
descriptor z, $\hat{z}$	reuse(k) and spread(k) over the 13 intervals; measured (z) or predicted ($\hat{z}$)
coverage	share of charged references that the interpreter placed exactly
uncertain, unresolved	reference under data-dependent control / at a data-dependent address
batch, batch point	a set of loop iterations executed together as numpy arrays / one of them
skeleton	a hand-written numpy replay of a program's memory behaviour
gate	the idiom check that decides whether the static prediction is trusted
LOWO	leave one workload out: every program is held out in turn
blind	the model was committed before the workload it predicts was traced

3 The access-count spectrum decides α

3.1 Moments of a bin

Let the run reference address a (of s_a bytes) c_a times. A bin keeps each of the c_a references independently with probability $1/k$, so address a is in the bin unless all its references were dropped:

$$p_a = p(c_a, k) = 1 - \left(1 - \tfrac{1}{k}\right)^{c_a}. \tag{2}$$

The bin footprint is $m = \sum_a s_a X_a$ with independent indicators $X_a \sim \text{Bernoulli}(p_a)$, hence

$$\mathbb{E}[m] = \sum_a s_a\, p_a, \qquad \text{Var}[m] = \sum_a s_a^2\, p_a(1-p_a), \qquad \mathbb{E}[u] = \sum_a p_a, \qquad \text{truth} = \sum_a s_a. \tag{3}$$

We predict α_k by truth$/\mathbb{E}[m_k]$ and the spread by $\sqrt{\text{Var}[m_k]}$. The ratio of expectations differs from the expectation of the ratio by a second-order term in $\text{Var}[m]/\mathbb{E}[m]^2$, which is below 10^{-3} for all bins in our data except the sparsest bins of the smallest inputs. The spectrum is all that is needed: two programs with the same spectrum have the same α curve, whatever their code.

Proposition 1. *For any spectrum, $1 \le \alpha_k \le k$, with $\alpha_k = k$ exactly when every address is referenced once and $\alpha_k \to 1$ when every address is referenced many more than k times.*

Proof. $p(1,k) = 1/k$ and $p(c,k)$ increases in c towards 1, so $1/k \le p_a \le 1$; hence truth$/k \le \mathbb{E}[m] \le$ truth. □

Figure 2 shows both effects. An address enters a bin with probability $1 - 1/e \approx 0.63$ when it is referenced $c = k$ times; the transition from "almost never" to "almost always" spans about two decades of c around k. The α curve of a program is therefore a readout of how its bytes are spread over reference counts: the bytes referenced far fewer than k times dominate α_k.

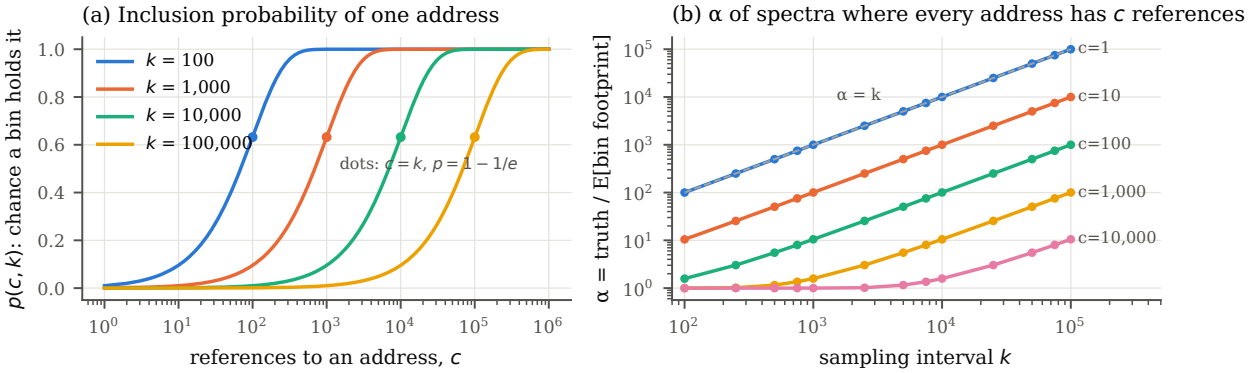


Figure 2: (a) The probability that a 1-in-k bin holds an address referenced c times, (2). (b) α_k for spectra in which every 4-byte address is referenced exactly c times: $\alpha_k \approx k/c$ while $c \ll k$, and $\alpha_k \to 1$ once $c \gg k$.

3.2 Assumptions

- **Independence.** The splitter draws each reference's inclusion independently (per bin, with its own pseudo-random stream), so the X_a are independent. A deterministic 1-in-k sampler (every k-th reference) would instead interact with the program's periodicity; MemPrint's spread term would then carry aliasing effects that the spectrum alone does not determine.

- **Time does not matter.** α of a whole-run footprint depends only on how often each address is referenced, not when. (Footprint over time, with memory freed, needs the timing as well; see the companion report [24].)
- **Address reuse.** When the allocator reuses an address range for a new block, the references to both blocks count for the same addresses. The interpreter and the skeletons therefore place heap blocks as glibc does (Section 4.7).

3.3 A descriptor of memory behaviour

For comparing programs we summarise the bin statistics at one config by a descriptor with two parts, each a curve over the 13 intervals:

$$\text{reuse}(k) = \frac{\log \alpha_k}{\log k}, \qquad \text{spread}(k) = \log \frac{\text{SD}_k}{\mathbb{E}[m_k]}. \tag{4}$$

reuse(k) is 1 when no address is referenced twice ($\alpha_k = k$) and falls towards 0 under heavy reuse; it is dimensionless and comparable across programs of very different size. spread(k) is the bins' coefficient of variation on a log scale. The descriptor $z(W) \in \mathbb{R}^{26}$ is *measured* from a program's traces; $\hat{z}(W)$ is *predicted* by evaluating (3) on the static spectrum plus the runtime baseline. The distance between two programs is the Euclidean distance after standardising every component with the mean and standard deviation of the known programs (missing components are set to the mean). We ask in Section 7 whether this distance predicts how well one program's model transfers to the other.

4 Computing the spectrum from source code

4.1 Overview

The interpreter (`analysis/memprint/static/interp.py`, about 1,500 lines of Python) parses the program with libclang [19] through its C API (ctypes), and executes the abstract syntax tree (AST) from `main()` without the program's input data. It is an abstract interpreter in the sense of [7]: values are either exact integers or the abstract value $\top$ (`UNK`, unknown), and every memory reference the program would make is *charged* to its address. The result is the per-address reference count, i.e. the spectrum, plus counts of what could not be placed.

4.2 Values and memory

- **Integers** (sizes, loop indices, flags) are tracked exactly as Python integers or numpy `int64` arrays. Conversions to C types wrap to the type's width and signedness (`wrap_int`).
- **Floating-point values** are unknown in the C interpreter; the C++ interpreter tracks floating-point *scalars* (Section 9.2).
- **Memory objects** (`Obj`): the stack, globals, `.rodata` and every heap block. Each keeps per-access-size reference counts per byte offset, so that counts can later be folded into 4-byte units, and, for irregular code, the integer values stored in it (one `int64` per 4-byte unit, plus a byte array for objects smaller than 4 bytes), with a sentinel for "unknown".
- **Pointers** are pairs (object, byte offset); the offset may be an array (one per batch point).

4.3 Batches: running a loop nest all at once

A counted loop (a `for` loop whose bound and step are known integers) does not run iteration by iteration. The interpreter computes its trip count for every enclosing batch point and runs the body once for all iterations together: the loop variable becomes a numpy array, and every expression in the body is evaluated element-wise. A statement in a nest of depth d is thus executed once, with one batch point per iteration of the nest. Batches are processed in chunks of at most 2^{21} points (`CHUNK`). Each batch point carries a weight w, so a statement charges $\sum w$ references; weights below 1 express uncertainty (below). This makes MEDIUM PolyBench inputs (up to 3.6×10^9 references) take at most 36.59 s (Figure 20a).

Loops that cannot be batched.

- **General loops** (`while`, `do`, `for` with an unknown bound) run one iteration at a time for all batch points still in the loop: each iteration evaluates the condition per point and drops the points for which it is false (`_loop_general`). A cap of 2^{24} iterations guards against runaway loops.
- **Loops carried through memory**: a loop whose body reads and then writes an integer array (a prefix sum, a BFS claiming vertices) gives different values if its iterations are batched. If it is not nested in another such loop and has at most 2×10^6 iterations (`SEQUENTIAL_LIMIT`), it runs one iteration at a time; otherwise it stays batched, and the values it computes may differ from a sequential run.
- **Repeated indices**: `a[i]++` and `a[i] += x` in a batched loop keep their sequential meaning when several batch points hit the same element (degree counting, `nbr[pos[u]++] = v`): the

updates are applied in iteration order with a stable sort and a cumulative sum per element (`_rmw_add`); `|=`, `&=` and `^=` with repeated offsets reduce with `ufunc.at`.

- **Loops of a few rounds**: a loop of at most 16 iterations (`FEW_TRIPS`) whose body contains a loop, directly or in a function it calls, runs one iteration at a time; its inner loops are still batched. Batching it saves little, and its rounds often depend on each other through memory in ways the syntactic check misses (GAP cc's two neighbour rounds, each linking components the previous one compressed).

4.4 Symbolic counting of affine innermost loops

A batch still evaluates every iteration: its cost grows with the program's references, like the program itself (a median of 25× native at MEDIUM, Section 7.7). For the innermost loops of affine code it does not have to. When a counted innermost loop's body (i) indexes arrays only with affine functions of the loop variable, (ii) stores no value that a later iteration or a branch needs, (iii) has no inner loop, break or call that could write memory, and (iv) has a scalar step, the interpreter runs the body *once*: the loop variable becomes an affine value $v = b + c\,i$ for $i = 0..t-1$ (`Aff`, with b per enclosing batch point), each scalar reference is charged with weight $w \cdot t$, and each array reference adds one count to every element of the arithmetic progression $\{o + s\,i\}$ it touches. Progressions are added with difference arrays per (element size, stride) and folded into the counts lazily, so the cost is that of the body once plus the touched range, not of t iterations. Floating-point values computed in such a loop are not needed (they decide no branch and no address, by (ii)).

Identity. For all 189 (kernel, config) pairs of PolyBench, the spectrum, the reference count and the checksum of the computed values are identical with and without symbolic counting (`paper_generalization/symbolic.py`).

Speed. The median speed-up grows with the input: 1.0× at MINI, 2.2× at SMALL2 and 5.6× at MEDIUM (up to 19×; Figure 3a). At LARGE and EXTRALARGE, where enumerating every iteration exceeds 900 s for most kernels, the interpreter takes 5–295 s, 0.16–1.6× the native run, for 9 of the 10 kernels measured (Figure 3b). The exception is atax (140× native): its time is in initialisation loops that store computed values element by element, which (ii) excludes. Symbolic counting is used by the C interpreter; the C++ interpreter, whose loops in GAP are irregular, enumerates.

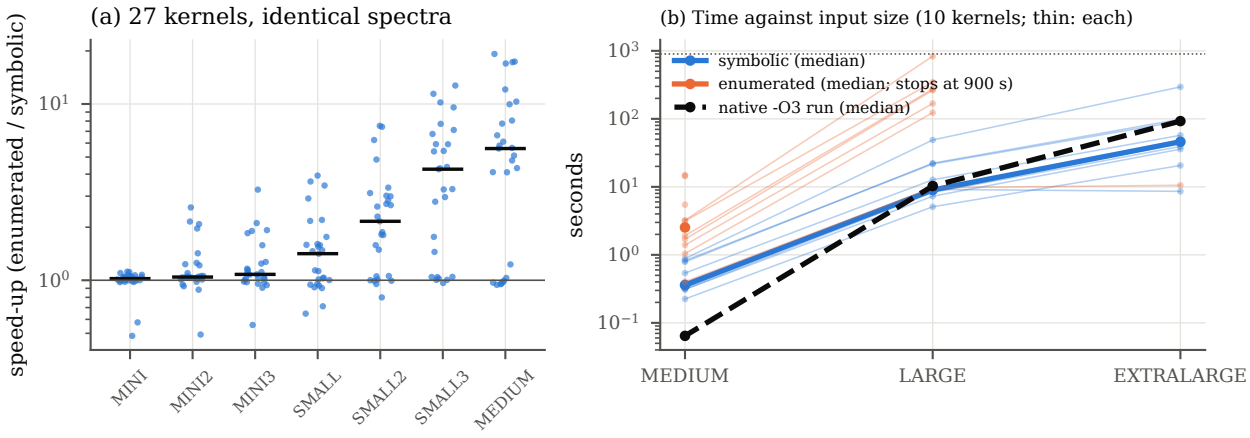


Figure 3: (a) Speed-up of symbolic counting over enumerating every iteration, per PolyBench kernel and config; the spectra are identical. (b) Interpreter time against input size for ten kernels, symbolic and enumerated (stopped at 900 s), with the native run.

4.5 What is charged (`-O0`)

PolyBench is traced at `-O0`, where every variable lives in memory. The C interpreter therefore charges: reads and writes of locals and parameters (stack slots, at the offsets a frame layout pass assigns; frames at the same call depth share addresses, `FRAME_BYTES` = 8192), array elements, floating-point constants (each distinct constant gets a `.rodata` slot), and each call's return address and saved frame pointer. Local arrays larger than 1,024 bytes get their own object. The interpreter charges a median of 87.32% of the references Pin observes (Figure 20b); the remainder is additional `-O0` stack traffic of gcc (spills, temporaries) on a few addresses that are referenced so often that every bin samples them, which leaves α unchanged.

4.6 Uncertainty and coverage

The interpreter does not guess. A branch whose condition is unknown executes both arms, each with half the weight, and marks their references *uncertain*; a loop with an unknown trip count runs its body once (`GUESS_TRIPS` = 1), uncertain. A reference whose address depends on unknown data is *unresolved*: it is counted but not placed. *Coverage* is the share of references that are neither. All PolyBench kernels reach a coverage of 1.0 since integer values in memory are tracked; before that, floyd-warshall and nussinov, whose branches compare values computed in memory, had 0.77.

4.7 Irregular C: memory values, random numbers, the heap

To run programs whose control depends on data they compute (graph codes), the interpreter also:

- remembers integer values stored in memory, so loops bounded by loaded values (`row[v+1]`) and subscripts that load an index (`A[B[i]]`) resolve;
- draws `rand()`, `random()` and `lrand48()` from their distributions with a seeded generator, and charges the C library's own references per call: a microbenchmark under Pin measured about 27 references per call, 24 to about 120 bytes of generator state and 3 to its 31-word table;
- places heap blocks as glibc 2.28 does (`skeleton.Process`, described in Section 8), so that addresses reused by later blocks are counted once.

4.8 Implementation constants

Table 2: Constants of the interpreters and skeletons.

Name	Value	Role
`UNIT`	4 B	counting granularity
`CHUNK`	2^{21}	batch points per chunk
`MAX_ITERATIONS`	2^{24}	iterations of a general loop before giving up
`SEQUENTIAL_LIMIT`	2×10^6	iterations a memory-carried loop may run one at a time
`FEW_TRIPS`	16	loops of at most this many rounds (with inner loops) run one at a time
`GUESS_TRIPS`	1	trips charged for a loop with unknown trip count
`FRAME_BYTES`	8192	stack frame size (`-O0`)
`STACK_ARRAY_LIMIT`	1024 B	larger local arrays get their own object
`REGISTER_BYTES`	64 B	C++ objects up to this size live in registers (`-O3`)
`MMAP_THRESHOLD`	128 KiB (dynamic, $\leq$ 32 MiB)	glibc's mapping threshold
`TOP_PAD`	128 KiB	heap growth pad
`MINSIZE`	32 B	smallest chunk
`HEAP_TOP_AT_START`	59,328 B	free bytes in the top chunk at a GAP program's first large allocation

5 The runtime baseline

A traced run also touches memory that the program text does not describe: the dynamic loader, C library initialisation, `stdio` buffers, allocator metadata. We model it as one spectrum shared by all programs of the same runtime: b_j bytes in n_j addresses referenced $c_j = 2^j$ times, $j = 0, \ldots, 26$. Its contribution to the moments is (3) summed over the 27 classes (addresses of a class have b_j/n_j bytes). The coefficients are fitted by non-negative least squares [17] to the residuals of the known programs: for every known program, config and interval, the observed mean bin footprint minus the static prediction,

$$\min_{b \geq 0} \sum_{(W,\mathrm{cfg},k)} \left(\frac{\bar{m}_{\mathrm{obs}} - \mathbb{E}[m]_{\mathrm{static}} - \sum_j b_j \, p(c_j, k)}{\bar{m}_{\mathrm{obs}}} \right)^2 ,$$

plus one row per config at $k = 1$ whose target is the true footprint. Residuals are relative so that small and large inputs weigh alike. The addresses n_j are fitted in the same way on unique-address counts. In the leave-one-out evaluation the baseline is refitted every time without the held-out program.

Fitted on all 27 PolyBench kernels, the baseline has 105 KB, almost all referenced once, plus about 3 KB referenced 32–512 times; for GAP's C++ runtime (libstdc++, `-O3`) it has 463 KB (Figure 4, Table 3). The baseline matters most for small inputs: it is half the footprint of GAP at scale 10.

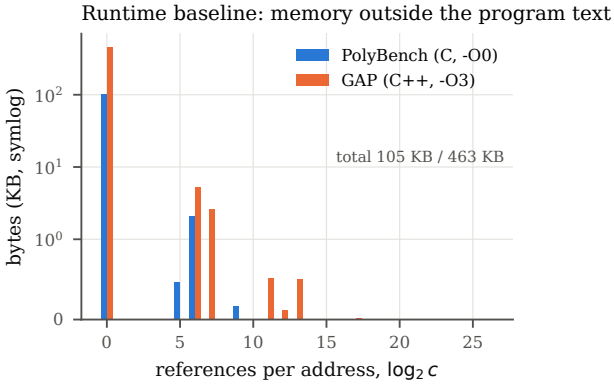


Figure 4: Runtime baselines fitted by NNLS: bytes per reference-count class.

Count class c_j	PolyBench		GAP	
	bytes	addresses	bytes	addresses
2^0	104,405	17,406	464,836	98,641
2^5	483	0	0	0
2^6	2,163	0	5,332	0
2^7	0	0	2,673	0
2^9	169	0	0	0
2^{10}	4	0	0	0
2^{11}	0	0	535	0
2^{12}	0	0	118	0
2^{13}	0	0	518	0
2^{17}	0	0	14	1
2^{18}	0	0	9	0
total	107,225	17,406	474,036	98,652

Table 3: Non-zero baseline classes.

6 Using the spectrum for an unseen program

This section defines the eleven ways of obtaining a footprint or a correction factor for an unseen program C that are compared in Section 7: what each computes, from what, and what it needs. Figure 5 summarises the requirements, Figure 6 the three families, and Table 4 the formulas.

6.1 The setting: what is known and what is asked

What is known.

- **The unseen program** C: its source code and the input configuration it will be run with (its *test config*). For PolyBench the config is a compile-time size (`-DMEDIUM_DATASET`); for programs with a command line it is the command line.
- **The known programs** $\mathcal{W}$ (in the evaluation, the other 26 PolyBench kernels). Each was traced completely (Pin splitter) at all seven configs, so for each $W \in \mathcal{W}$ we have its traces (true footprints and splitter bins), its own model M_W ((1) fitted on all its configs), its measured descriptor z_W ((4)), and its static spectra.
- **The runtime baseline** b, fitted on the known programs' traces (Section 5); it is refitted without C in every leave-one-out round.

What is asked. Either the true footprint of C at its test config, or the correction factor α_k that turns what a 1-in-k sample of C's references sees into the footprint:

$$\widehat{\text{truth}} = \hat{\alpha}_k \cdot m_k, \tag{5}$$

where m_k is the footprint of one bin of a *sampled run* of C: one Pin run of C at its test config that keeps one reference in k and splits the kept references into bins (MemPrint's sampler; Section 2). This is how MemPrint is used: a cheap sampled run, scaled up by a predicted α.

Why most methods need a sampled run. A regression model (1), whether C's own or borrowed, predicts α *from* the sampled run: its inputs are the bin footprint m_k, the interval k and the spread SD_k of m_k across the bins. Without a sampled run there is nothing to put into the model. Static α needs no input from a run to compute, but it is still a correction factor: turning it into a footprint needs an m_k to multiply ((5)). Only the static footprint needs no run of C at all. In the evaluation every α is scored against the measured $\alpha = \text{truth}/m$ of each of C's splitter bins at the test config (13 intervals $\times$ 20 bins).

6.2 Family A: predicted from the source

All three methods start with the interpreter run on C's source at the test config (Section 4), which gives the spectrum $S_C = \{(c_a, s_a)\}$: the bytes s_a of every address a and the references c_a it receives. Figure 7 follows gemm at MEDIUM through the computation.

Static footprint. The predicted footprint is the program's own bytes plus the runtime's:

$$\widehat{\text{truth}}_C = \sum_a s_a + \sum_j b_j. \tag{6}$$

Needs: C's source and test config, and the baseline (fitted once on other programs' traces). *Does not need:* any run of C, any model. *Output:* the footprint. *Scored by* its error against the true

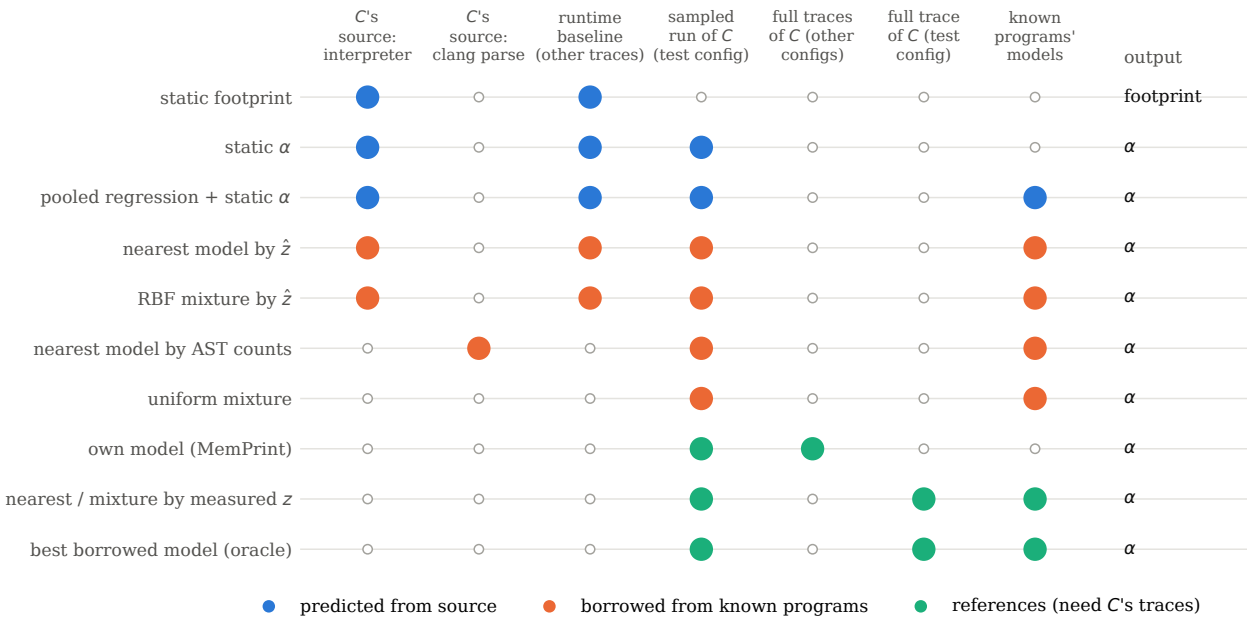


Figure 5: What each method needs (filled circles). Blue: predicted from C's source. Orange: borrowed from the known programs. Green: references that need traces of C itself. "Output" is what the method produces: the footprint directly, or a correction factor α to apply to a sampled run with (5). The costs of these requirements are measured in Section 7.7.

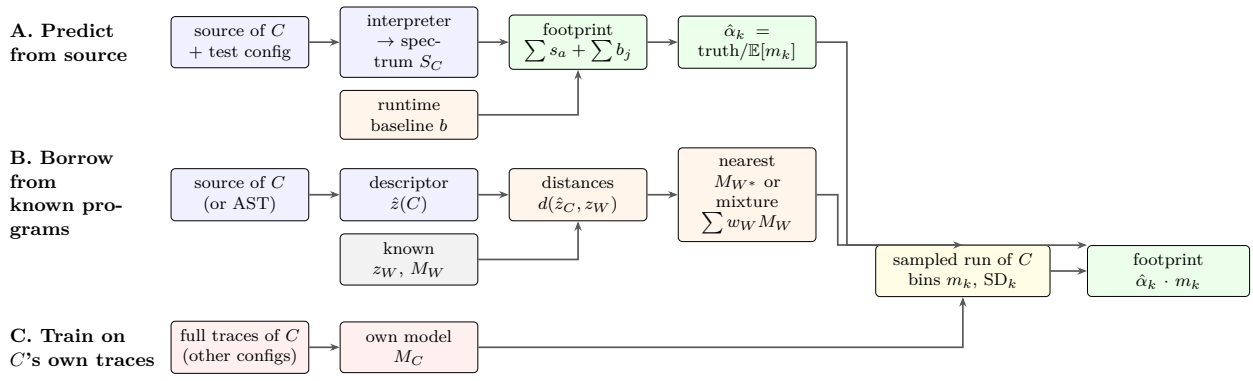


Figure 6: The three families. A: the spectrum computed from C's source gives the footprint directly, and α in closed form. B: a descriptor of C selects or weights the regression models of known programs. C: MemPrint's own model, trained on traces of C. The α of every family is applied to a sampled run of C; only the static footprint needs no run.

footprint of C's full trace. For gemm at MEDIUM the interpreter places 1,158,400 bytes (its three arrays exactly), the baseline adds 107 KB, and the prediction is 0.12% off.

Static α. The moments of (3) on S_C and the baseline give the expected footprint of a 1-in-k bin, and hence

$$\hat{\alpha}_k = \frac{\sum_a s_a + \sum_j b_j}{\sum_a s_a\, p(c_a, k) + \sum_j b_j\, p(2^j, k)}, \qquad p(c, k) = 1 - (1 - 1/k)^c, \tag{7}$$

for every interval k of the sampled run. Panel (b) of Figure 7 shows why α grows with k: at $k = 100$ most of gemm's bytes, referenced a few hundred times each, are in every bin; at $k = 10^5$ almost none are. *Needs:* the same as the static footprint, plus a sampled run of C to apply $\hat{\alpha}_k$ to ((5)). *Does not need:* any model or training. *Output:* $\hat{\alpha}_k$ for each k.

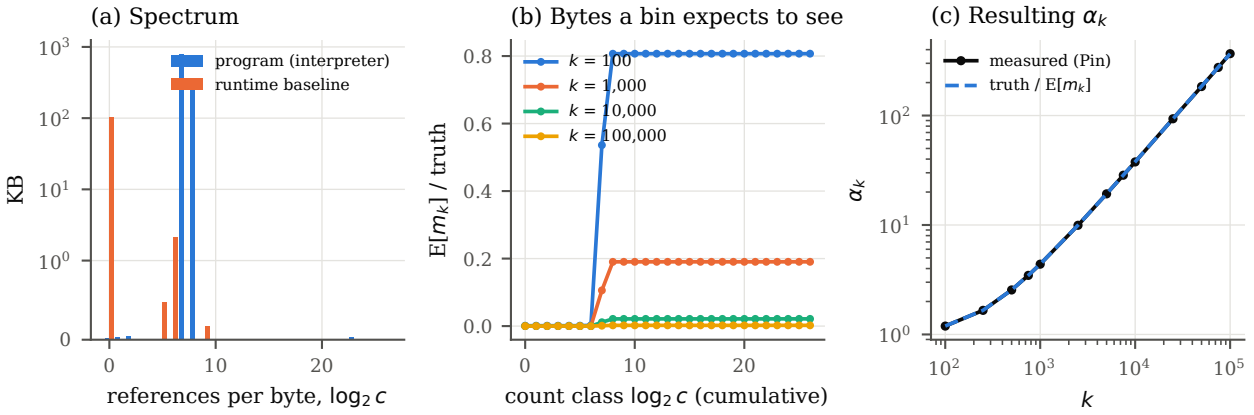


Figure 7: The static route for gemm at MEDIUM. (a) The spectrum: bytes per reference-count class, for the program (from the interpreter) and the runtime baseline. (b) The share of the true footprint that a 1-in-k bin is expected to contain, accumulated over the count classes ($\sum s_a p(c_a, k)$ divided by the truth): rarely referenced bytes contribute little at large k. (c) $\hat{\alpha}_k$ from (7) against the measured truth over the mean bin footprint.

Pooled regression with static α. One regression of the form (1) is fitted over the bins of *all* known programs at all their configs, with $\log \hat{\alpha}_k$ of each bin's program and config (from its static spectrum) as an eighth feature:

$$\log \alpha = b_0 + \sum_{i=1}^{7} b_i\, x_i(m_k, \mathrm{SD}_k, k) + b_8 \log \hat{\alpha}_k, \tag{8}$$

where $x_1, \ldots, x_7$ are the seven features of (1). It is then applied to C's sampled bins with C's static $\hat{\alpha}_k$. The idea is that the regression learns how far the closed form is off. *Needs:* the interpreter on C, a sampled run of C, and the known programs' traces and spectra. *Output:* α. It is less accurate than static α alone (Section 7): the other features bring in how the known programs' α varies with m and SD, which does not transfer.

6.3 Family B: borrowed from known programs

These methods do not predict α themselves; they choose or weight the regression models M_W of known programs, and apply them to C's sampled run.

The descriptor and the distance. C's descriptor $\hat{z}(C)$ is (4) evaluated on the moments of its static spectrum plus the baseline: 13 values of reuse(k) and 13 of spread(k). Each known program's z_W is measured from its traces at the config of the same name (PolyBench's sizes are the same names for every kernel). Every component is standardised with the mean and standard deviation of the known programs, and

$$d(\hat{z}_C, z_W) = \Big\| \frac{\hat{z}_C - \mu}{\sigma} - \frac{z_W - \mu}{\sigma} \Big\|_2. \tag{9}$$

Nearest model by $\hat{z}$. The model of the closest known program is applied to C's bins: $W^* = \arg\min_W d(\hat{z}_C, z_W)$ and $\log \hat{\alpha} = M_{W^*}(m_k, \mathrm{SD}_k, k)$. *Needs:* the interpreter on C, a sampled run of C, the known models and descriptors.

RBF mixture by $\hat{z}$. Every known model's prediction is averaged in log space with Gaussian weights:

$$\log \hat{\alpha} = \sum_W w_W \, M_W(m_k, \mathrm{SD}_k, k), \qquad w_W = \frac{\exp(-d(\hat{z}_C, z_W)^2/2h^2)}{\sum_V \exp(-d(\hat{z}_C, z_V)^2/2h^2)}, \tag{10}$$

with the bandwidth h the median, over the known programs, of each one's distance to its nearest other known program (0.81 in the example below). Close programs dominate; distant ones get almost no weight. *Needs:* as the nearest model.

Nearest model by AST node counts. The similarity baseline: each program is described by the number of nodes of every kind in its clang AST, as $\log(1 + \text{count})$, standardised like z; the nearest program's model is applied. *Needs:* a parse of C (no interpreter), a sampled run of C, the known models and their AST counts. It tests whether syntax alone tells which model transfers.

Uniform mixture. All known models with equal weights $w_W = 1/|\mathcal{W}|$: borrowing with no notion of similarity. *Needs:* a sampled run of C and the known models.

Figure 8 shows the borrowing methods on atax with its largest input held out. Its static descriptor lies almost on top of its measured one, next to mvt and bicg, whose models are 6–7% off on atax, while most known programs' models are 40–90% off (panel a). Distance in $\hat{z}$ ranks the transfer error well ($\rho = 0.89$), as the measured descriptor does, while AST distance does not ($\rho = 0.35$; panel b). The mixture puts 85% of its weight on mvt and bicg (panel c). The resulting errors: nearest model 6.7%, mixture 5.5%, against 6.0% for the oracle and 4.2% for static α; atax's own model has 19.5%.

6.4 Family C and the references

These need traces of C itself, so they are not available for an unseen program; they bound what the others can do.

Own model (MemPrint). M_C is (1) fitted on the splitter bins of C's other six configs (all intervals), and applied to C's sampled bins at the test config. This is MemPrint as published: train on full traces of small inputs of the same program, then predict a larger one (EXTRA) or one in between (INTER). *Needs:* six full traces of C and a sampled run.

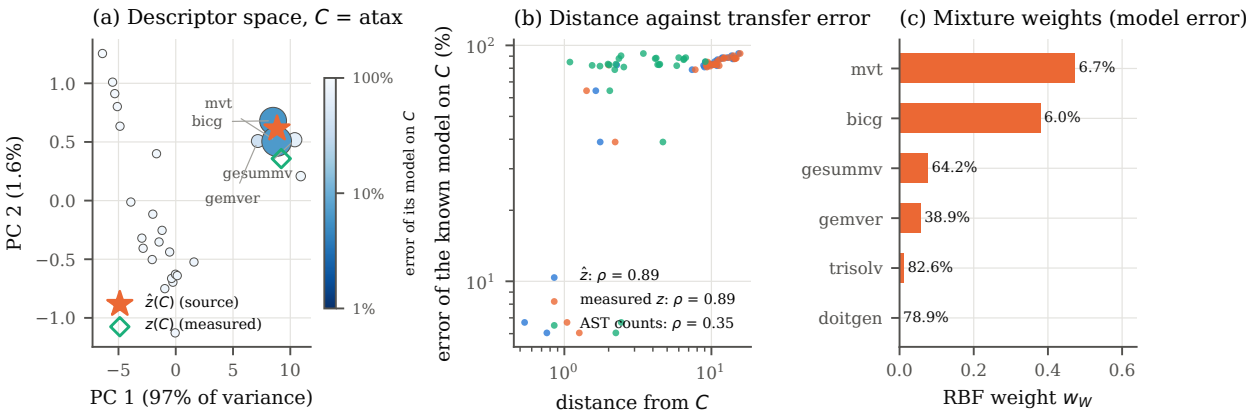


Figure 8: Borrowing for atax (EXTRA). (a) The 26 known kernels in the first two principal components of the standardised descriptors, coloured by the error of their model on atax and sized by their RBF weight; the star is atax's descriptor predicted from source, the diamond its measured one. (b) Distance from atax against the error of each known model on atax, for the three distances, with Spearman's ρ. (c) The six largest mixture weights, with each model's error on atax.

Nearest model and mixture by measured z. The same as their $\hat{z}$ versions, with C's descriptor measured from its splitter bins at the test config instead of predicted. They show how much is lost by predicting the descriptor; they are not usable methods, because the full trace they need already gives the exact footprint.

Oracle. The known model with the lowest error on C, chosen after scoring all 26 with the truth: the best that choosing one borrowed model could ever do.

Table 4: The methods at a glance. "Needs of C" is what the method needs of the unseen program itself; all methods but the static footprint and the own model also use the known programs.

Method	What it computes	Needs of C	Output
Static footprint	$\sum_a s_a + \sum_j b_j$ (6)	source	footprint
Static α	truth/$\mathbb{E}[m_k]$ of $S_C + b$ (7)	source; sampled run to apply	α_k
Pooled regression	(1) + $\log \hat{\alpha}$, fitted on all known bins (8)	source; sampled run	α
Nearest by $\hat{z}$	M_{W^*}, $W^* = \arg\min d(\hat{z}_C, z_W)$	source; sampled run	α
Mixture by $\hat{z}$	$\exp \sum_W w_W \log M_W$, Gaussian weights in d	source; sampled run	α
Nearest by AST	M_{W^*} by distance in AST node counts	parse; sampled run	α
Uniform mixture	$\exp \frac{1}{\lvert\mathcal{W}\rvert} \sum_W \log M_W$	sampled run	α
Own model	(1) fitted on C's six other configs	6 full traces; sampled run	α
By measured z	as by $\hat{z}$, with $z(C)$ from its trace	full trace; sampled run	α
Oracle	the known model best on C	the truth	α

6.5 The access-idiom gate

The static prediction is trustworthy where the interpreter places the references, i.e. for code close to the affine kernels it was validated on. A separate pass over the AST (`static/idioms.py`) classifies

every memory reference by how its address is formed: *affine* (subscripts built from integer variables, constants, +, − and multiplication by a constant), *indirect* (a subscript that loads memory or calls a function: `A[B[i]]`, CSR neighbours, hash lookups), *pointer* (dereference or `->`) or *other*; and every loop as *counted*, *data-bounded* (bound is a load that depends on an enclosing loop variable, `e < row[v+1]`) or *while*. References are weighted by $10^{\text{loop depth}}$ as a static stand-in for their dynamic count. The gate sends a program to training-free sampling if the interpreter's coverage is below 0.5, the affine share below 0.95, or more than 5% of loops are data-bounded or while loops.

7 Evaluation on PolyBench

7.1 Protocol

Every kernel C is held out in turn (leave one workload out, LOWO; `analysis/memprint/lowo.py`). Only C's source is used to predict; C's traces are used only for scoring and for the reference methods that are marked as using them. The 26 other kernels provide the baseline, their own models (fitted on all their configs) and their measured z (at the config of the same name as C's test config). The test config is MEDIUM (EXTRA) or SMALL2 (INTER). Static spectra for all 27×7 kernel–config pairs are computed once (`python -m memprint static spectra`).

7.2 Accuracy

Table 5: Leave-one-workload-out MAPE (%) of α on the held-out kernel, 27 kernels. The first row is the footprint error. "Uses" says what each method needs beyond the known programs.

Method	Uses	EXTRA		INTER	
		median	mean	median	mean
Static footprint (source + runtime baseline, no sampling)	source	0.09	0.12	0.27	0.49
Static α (moments of the static spectrum)	source	1.19	2.66	5.50	6.59
Pooled regression with log static α as a feature	source + known traces	4.10	6.83	6.13	7.33
Nearest known model by $\hat{z}$	source + known models	12.24	16.97	11.11	15.09
RBF mixture of known models by $\hat{z}$	source + known models	14.07	19.34	12.11	15.43
Nearest known model by measured z	C's traces (upper bound)	13.84	19.45	11.19	16.01
RBF mixture by measured z	C's traces (upper bound)	14.05	18.57	11.53	15.61
Nearest known model by AST node counts	source + known models	26.01	144.61	17.44	34.83
Uniform mixture of known models	known models	92.03	154.16	43.99	79.80
C's own model, trained on its other inputs	C's traces	18.04	18.41	10.85	11.22
Best known model, chosen after scoring	C's traces (oracle)	8.89	11.72	6.05	9.20

Table 5 and Figure 9 give the results; Table 23 in the appendix lists every kernel.

- **Predicting beats borrowing.** Static α is within a median of 1.19% (EXTRA) and 5.50% (INTER). The kernel's own model, trained on its own traces at smaller inputs, has 18.04% and 10.85%, and even the oracle, the best borrowed model picked after seeing the errors, has 8.89% and 6.05%. Figure 10 shows that the source beats the own model on almost every kernel under EXTRA, where the own model has to extrapolate.
- **The static footprint is nearly exact** (median 0.09% / 0.27%; Figure 11). The worst cases are correlation, gramschmidt and cholesky at their INTER size (1.7–2.6%).
- **The pooled regression** with static α as a feature is worse than static α alone: the regression's other features add the known programs' idiosyncrasies.
- **Similarity helps when borrowing**: choosing a model by $\hat{z}$ cuts the error of the uniform mixture by a factor of 4–8.

7.3 The moments match Pin

Figure 12 overlays the predicted α_k curve on every bin Pin recorded, for nine kernels at MEDIUM, and Figure 13 compares all three predicted moments with the measured ones over every kernel, config and interval. The mean bin footprint is predicted within a median of 0.85%. The spread is

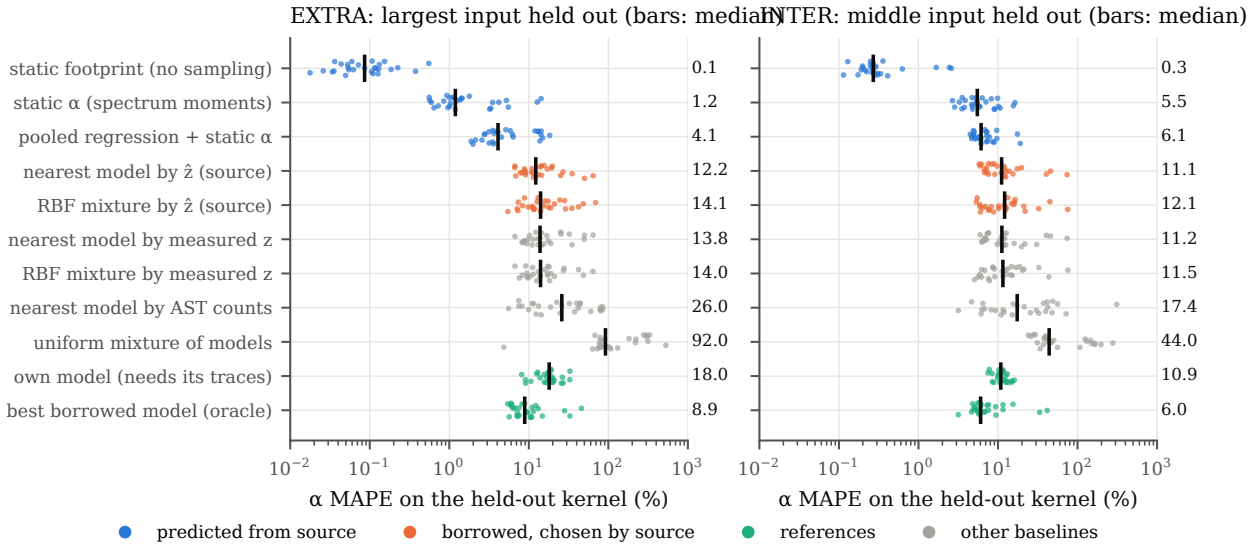


Figure 9: Leave-one-out MAPE of every method on each held-out kernel (dots; bars and numbers: medians). Blue: predicted from the source. Orange: a known model chosen by $\hat{z}$. Green: references that need the held-out kernel's traces. Gray: other baselines.

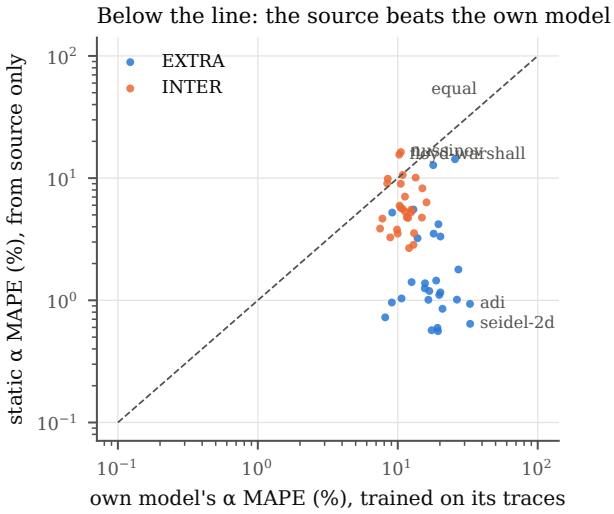


Figure 10: Static α error (from the source only) against the own model's error (trained on the kernel's traces), per kernel and split.

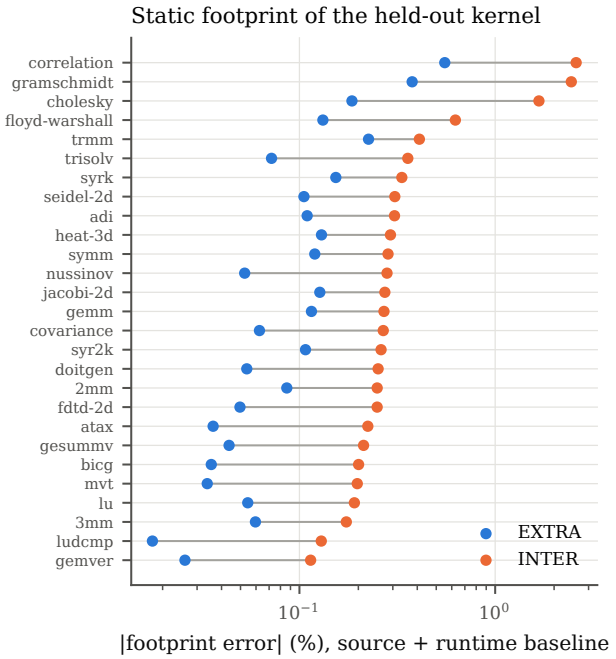


Figure 11: Footprint predicted from the source plus the runtime baseline, per held-out kernel.

predicted within a median of 13.82%, systematically a little low (Figure 15): with $B = 20$ bins the sample standard deviation itself has a relative standard error of about 16%, so most of the scatter is the measurement's. The unique-address count agrees except for floyd-warshall and nussinov, where the interpreter's unit granularity and Pin's start-address count differ by a factor of about 4; α does not use it.

Figure 14 shows the static α error at every size: it is largest at the smallest inputs, where the runtime baseline is a large part of the footprint, and for the two kernels with data-dependent `min`/`max` ternaries, floyd-warshall and nussinov (12.8–16.3%). Their outer loops now run sequentially, but their inner loops stay batched, so the values that decide the branches may differ from a sequential run.

7.4 The footprint over time from source

The spectrum gives the footprint at the end of a run; the same interpreter also gives it *over* the run. In timeline mode each batch point carries the interval of the outermost batched loop it belongs to, as a fraction $[t_{lo}, t_{hi})$ of that loop's time; each byte records the time of its first touch, and heap events carry the interpreter's reference clock. The program's live footprint at clock u is then the bytes first touched before u, less those freed. Pin's clock counts references the interpreter does not (the runtime's start-up and the `-O0` stack traffic), so the prediction maps the clocks linearly and adds the runtime:

$$\widehat{F}(t) = R(t) + F_{\text{prog}}\left(\frac{t - R_s}{s}\right), \tag{11}$$

where R_s (the start-up references) and s (Pin's references per interpreter reference) are the medians of per-kernel least-squares fits of Pin's total references against the interpreter's over the seven sizes, on the other 26 kernels; $R(t)$ is the runtime baseline B once the program has started, and during start-up B times the median shape of the other timeline kernels' start-up ramps. Nothing of the predicted kernel's traces is used.

Against Pin's exact live-footprint timelines of 2mm, atax, gemm and jacobi-2d at seven sizes (Figure 17, `paper_generalization/timeline_static.py`), the per-snapshot error has a median of 1.8% (median over the 28 runs; mean 5.1%), and the peak footprint is within 3.5% in every run (median 1.8%). Most of the mean error is on the time axis: the predicted run length is off by -8 to $+9$% (the clock fit), and on time normalised by each run's length the median mean error falls to 2.5%. Timeline mode cannot use symbolic counting (first touches need each iteration's time) and costs 23× the symbolic interpreter at MEDIUM (median, Table 20).

7.5 Dependence on the runtime baseline

The static footprint is not purely static: the runtime baseline (Section 5) is fitted on traces of *other* programs that share the compiler, C library and harness. Table 6 shows how much the prediction leans on it and how many traced programs it needs, refitting it, for every held-out kernel, on 1 or 3 other kernels drawn at random (five draws each) or on all 26 others (`baseline_sensitivity.py`).

- **The baseline is essential**: without it the median error is 47.19% (90th percentile 87.67%). It is 89% of the true footprint at MINI, 30% at SMALL2 and still 8.5% at MEDIUM, so it matters most for small inputs.
- **One traced program nearly suffices**: a baseline fitted on a single other kernel gives a median error of 0.08% (90th percentile 2.76%, worst 8.20%). More programs mainly shrink the tail (all 26: 90th percentile 0.63%, worst 4.66%); the median is slightly higher with 26, as one fit has to suit all of them.

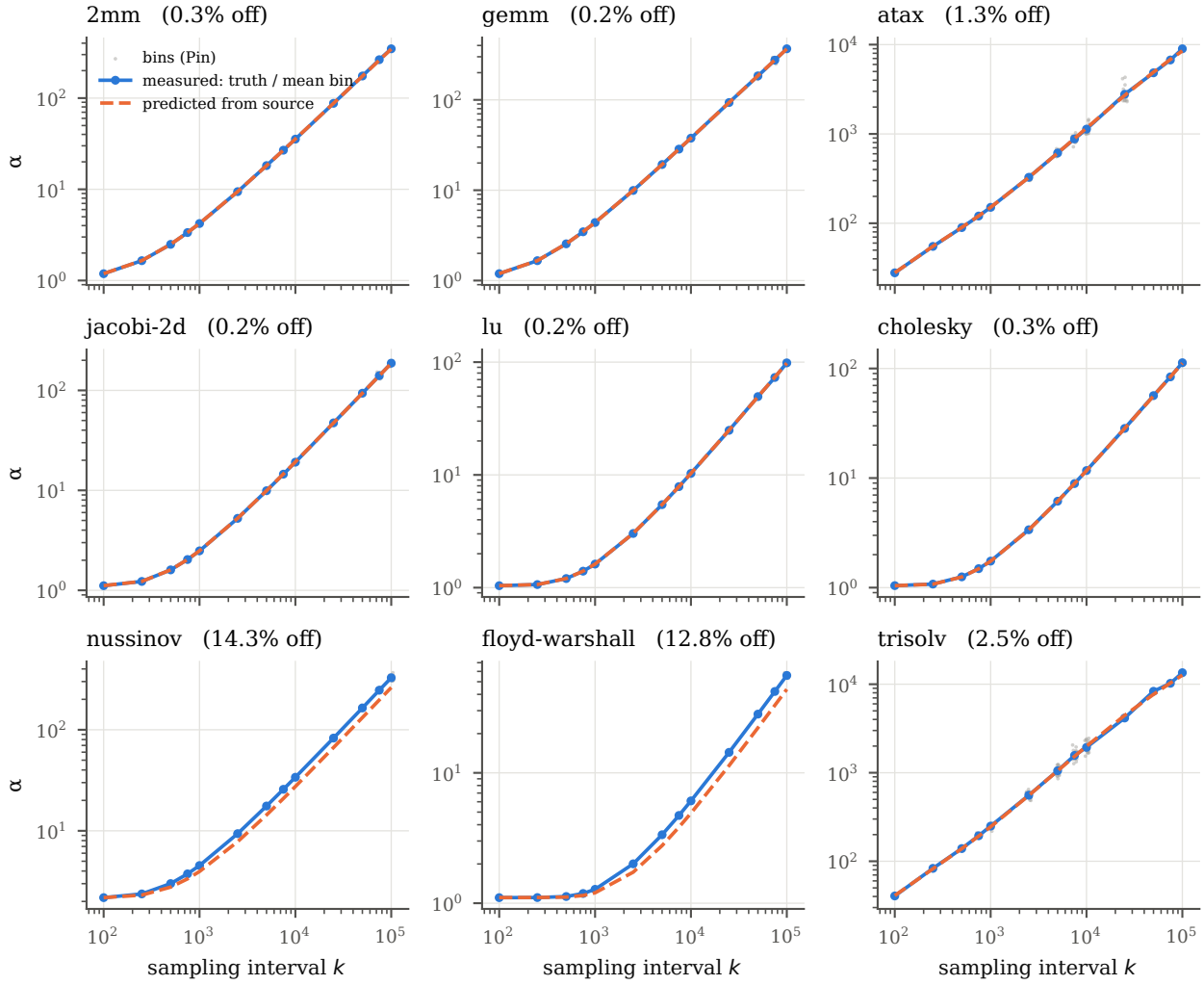


Figure 12: α against the sampling interval at MEDIUM: every Pin bin (gray), the measured truth over the mean bin footprint (blue) and the prediction from the source (orange, dashed). The title gives the MAPE over the 13 intervals.

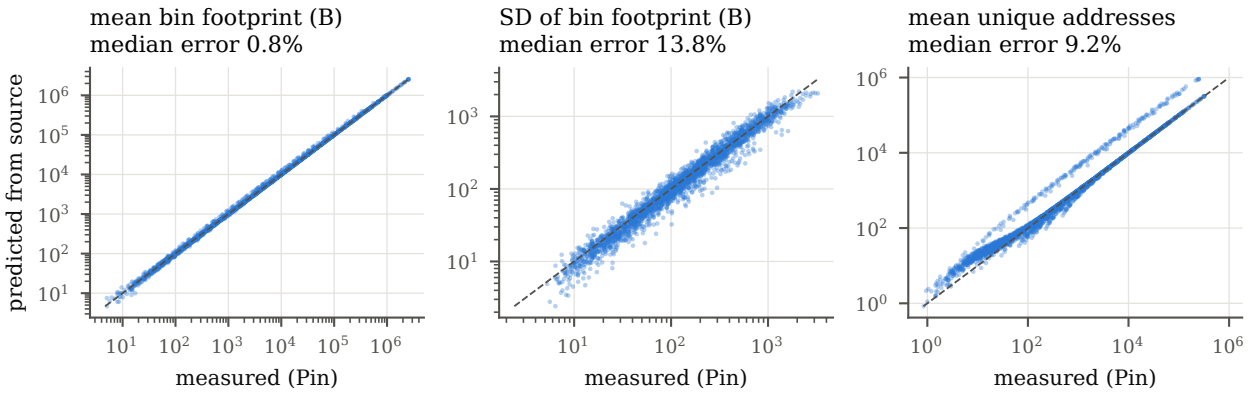


Figure 13: The three moments of (3), predicted from source against measured, for every kernel, config and interval (2,457 points).

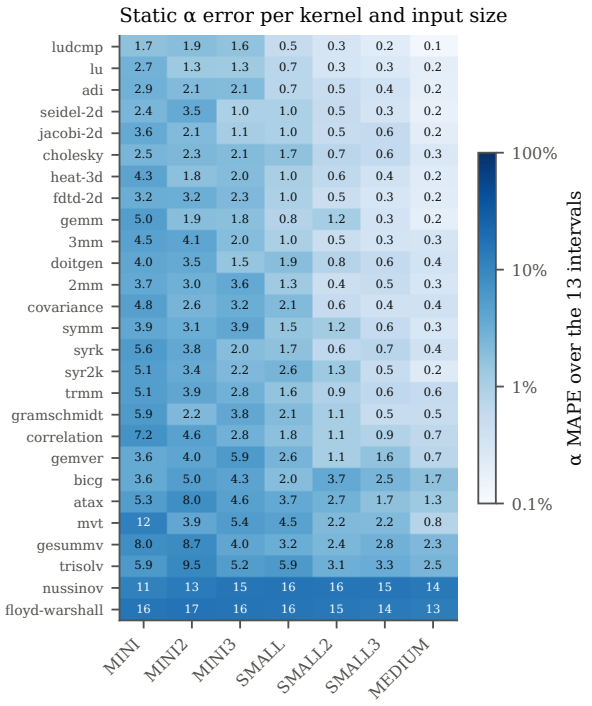


Figure 14: Static α MAPE per kernel and input size (baseline fitted without the kernel).

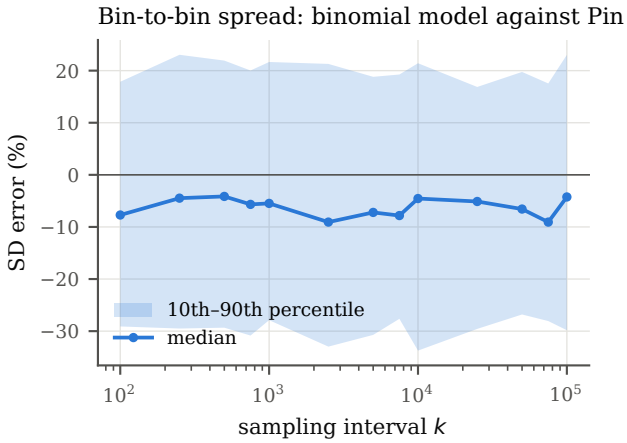


Figure 15: Signed error of the predicted standard deviation of the bin footprint, by interval.

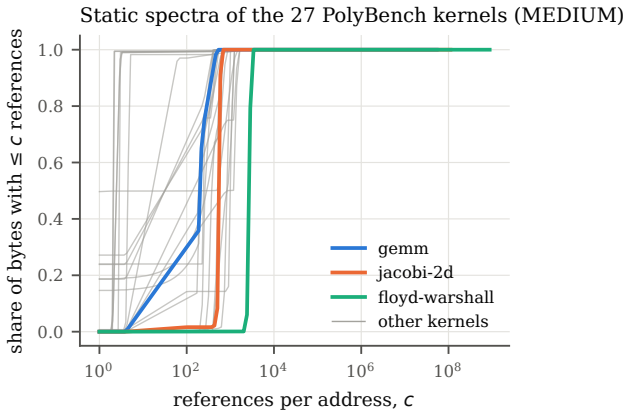


Figure 16: Cumulative static spectra at MEDIUM: the share of bytes referenced at most c times.

Runtime baseline fitted on	median	90th percentile	max
no baseline	47.19	87.67	94.17
1 other kernel	0.08	2.76	8.20
3 other kernels	0.09	1.58	5.06
all 26 others	0.29	0.63	4.66

Config	median truth (KB)	baseline share	error, no baseline	error, 1 kernel	error, 26
MINI	118	88.5%	88.0%	0.12%	0.50%
MINI2	133	78.9%	78.5%	0.10%	0.45%
MINI3	161	64.9%	64.8%	0.09%	0.36%
SMALL	225	46.6%	46.9%	0.06%	0.27%
SMALL2	349	30.0%	29.9%	0.05%	0.18%
SMALL3	615	17.0%	17.0%	0.09%	0.15%
MEDIUM	1235	8.5%	8.4%	0.06%	0.09%

Table 6: Static footprint error (%) over all 27 kernels and seven configs, each kernel held out, by what the runtime baseline is fitted on (left), and by input size (right): the median true footprint, the baseline's share of it, and the median error without a baseline, with one fitted on one other kernel, and on all 26.

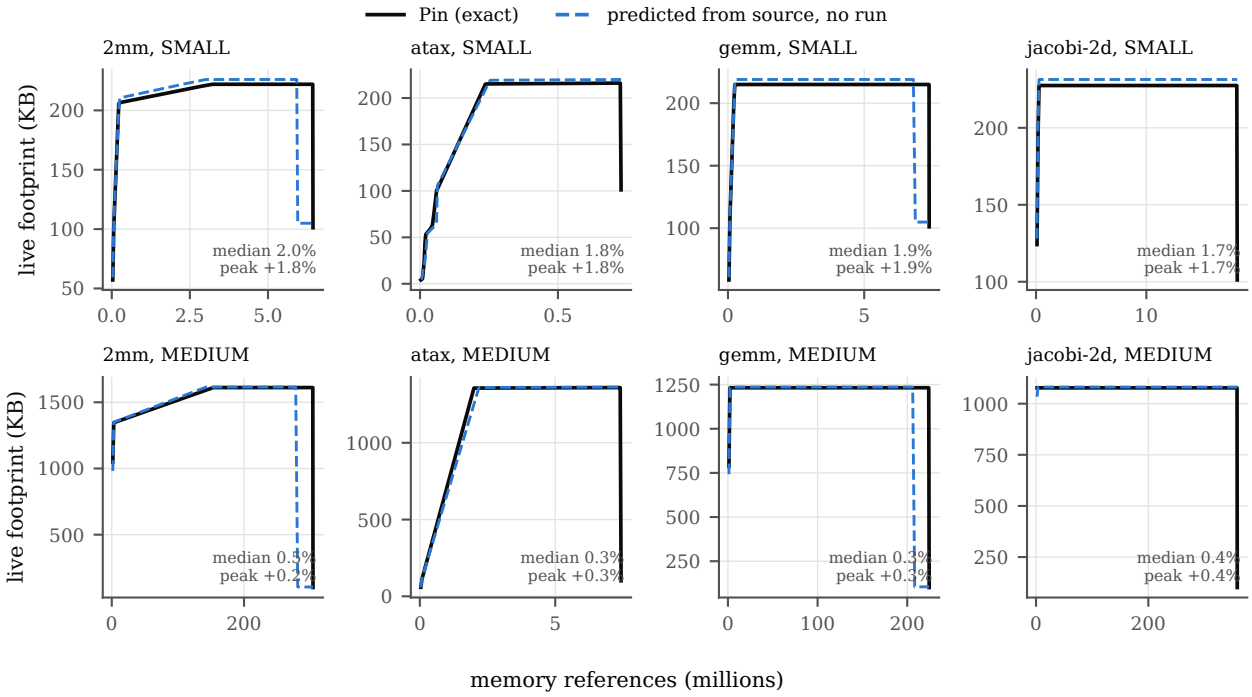


Figure 17: Live footprint over Pin's reference clock: Pin's exact timeline and the prediction from source with no run of the kernel, at SMALL and MEDIUM.

- **The cost is one trace per runtime, not per program.** The smallest full trace (a kernel at MINI) takes 0.91 s. A program with another runtime needs its own calibration traces: libstdc++ at `-O3` has a baseline of 463 KB (fitted on GAP), and MPI or a language runtime would need theirs. In the blind test (Section 10), GAP's baseline transferred to two other C++ programs with footprint errors within 2.1% and 4.2%; their own I/O (iostream, report files) is not part of it.

7.6 Similarity: does $\hat{z}$ rank transferable models?

For each held-out kernel we compute the Spearman rank correlation [31] between the distance from C to each known kernel and the error of that kernel's model on C. A high ρ means that nearer programs have models that transfer better.

Table 7: Spearman ρ between descriptor distance and transfer error, over the 27 held-out kernels.

Distance	EXTRA		INTER	
	median ρ	mean ρ	median ρ	mean ρ
$\hat{z}$ predicted from source	0.826	0.784	0.849	0.786
measured z	0.843	0.801	0.854	0.781
clang AST node counts	0.268	0.243	0.307	0.234

$\hat{z}$ ranks the transferable models as well as the measured z does, and much better than AST node counts (Table 7, Figure 18a). This answers the first question of the introduction: the spectrum-derived descriptor is a usable similarity, and it can be computed from code. $\hat{z}$ is also close to z in absolute terms: the median RMSE of reuse(k) is 0.0014, at most 0.032 (Figure 18b, Figure 19).

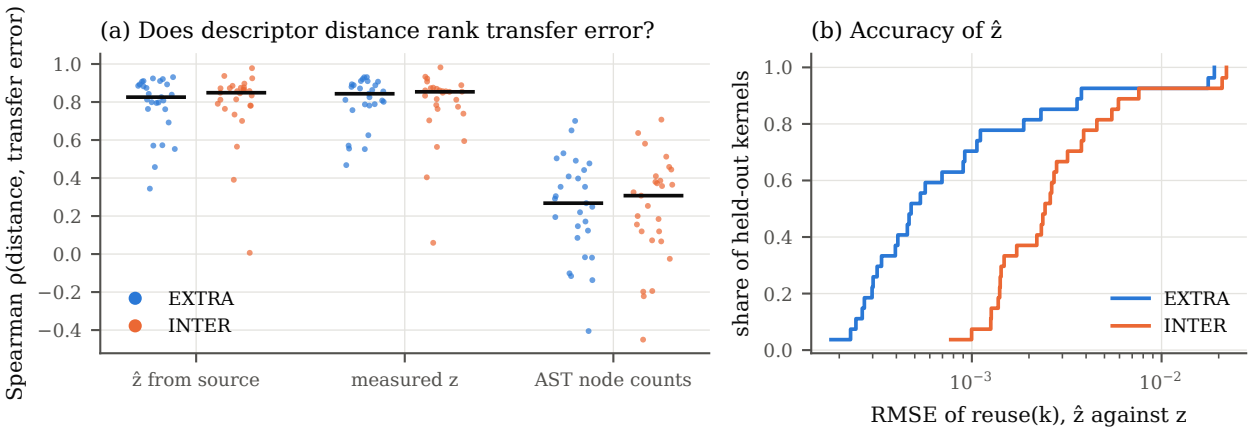


Figure 18: (a) Spearman ρ between descriptor distance and transfer error, one dot per held-out kernel (bars: medians). (b) Cumulative distribution of the RMSE between $\hat{z}$ and z on reuse(k).

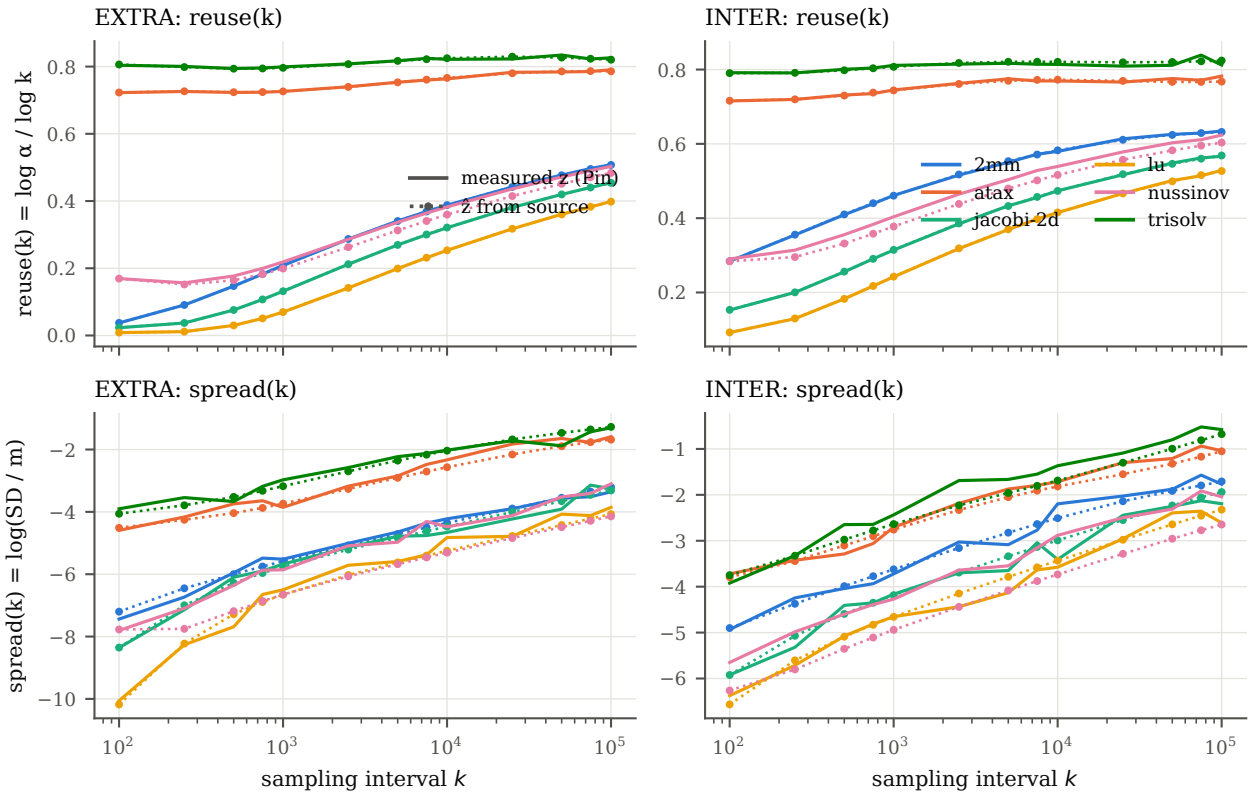


Figure 19: The two parts of the descriptor, measured (solid) and predicted from source (dotted), for six kernels.

7.7 Cost of each method

To compare the methods of Table 5 on cost as well as accuracy, we measured what each needs for the held-out kernel C at its test config (Table 8; `paper_generalization/overhead.py`):

- the *full trace* (Pin splitter at the test config), which gives the exact footprint;
- the *sampled run*, the run MemPrint deploys: Pin's sampler with `run.sh`'s defaults (`-i 1000 -s 100 -r 10`). Every method that applies a regression model, its own or a borrowed one, needs it, because the model predicts α from that run's bin footprint and spread;
- the interpreter (bytes touched, symbolic counting, Section 4.4) and a clang parse;
- for the own model, the full traces of C's six other configs it is trained on.

The analysis that turns these into a prediction (fitting the baseline, the moments, the descriptor, fitting and applying models) takes milliseconds to 0.4 s (Table 21) and is not counted. The methods that borrow known models, the pooled regression and the runtime baseline also need full traces of other programs once: 3,707 s for all 27 kernels at all seven configs. Every time in this section was measured afresh for this version, one run at a time, on a shared 32-core node (median 1-minute load 8.97; Section 12 describes the measurement and checks it against the times used before).

Table 8: Cost of each method of Table 5 for the held-out kernel at its test config: median over the 27 kernels of the seconds, of the ratio to the native run, and of the ratio to a full trace, with the median MAPE of Table 5 (%). "Runs of C" is what the method needs of the held-out kernel itself.

Method	Runs of C	EXTRA (MEDIUM)				INTER (SMALL2)			
		s	×native	×trace	MAPE	s	×native	×trace	MAPE
Native run (reference)	–	0.06	1	0.00	–	0.01	1	0.00	–
Pin, instrumentation only (reference)	–	2.76	41	0.06	–	1.85	166	0.28	–
Full trace, Pin splitter (exact footprint)	the full trace	45	586	1.00	0	6.72	540	1.00	0
Static footprint	interpreter	0.17	3	0.00	0.09	0.10	10	0.02	0.27
Static α	interpreter + sampled run	3.41	54	0.09	1.19	2.05	203	0.32	5.50
Pooled regression + static α	interpreter + sampled run	3.41	54	0.09	4.10	2.05	203	0.32	6.13
Nearest model by $\hat{z}$	interpreter + sampled run	3.41	54	0.09	12.24	2.05	203	0.32	11.11
RBF mixture by $\hat{z}$	interpreter + sampled run	3.41	54	0.09	14.07	2.05	203	0.32	12.11
Nearest model by AST counts	parse + sampled run	3.29	44	0.07	26.01	1.96	171	0.29	17.44
Uniform mixture	sampled run	3.26	44	0.07	92.03	1.92	167	0.29	43.99
Own model (MemPrint)	6 full traces + sampled run	33	446	0.74	18.04	74	5765	10.88	10.85
Nearest model by measured z	full trace + sampled run	48	662	1.07	13.84	8.64	774	1.29	11.19
RBF mixture by measured z	full trace + sampled run	48	662	1.07	14.05	8.64	774	1.29	11.53
Best borrowed model (oracle)	full trace	45	586	1.00	8.89	6.72	540	1.00	6.05

- **The static footprint is the cheapest and the most accurate.** It runs nothing of C: with symbolic counting the interpreter takes a median of 0.17 s at MEDIUM (3× native, 0.5% of a full trace), a twentieth of MemPrint's one sampled run (3.26 s, 44× native). It is slower than the sampled run only for nussinov (both splits) and floyd-warshall (EXTRA), whose time is in loops that cannot be counted symbolically (data-dependent minima over computed values) and which it predicts least accurately; nussinov reaches the maximum, 412× native at SMALL2.
- **The own model is the most expensive usable method**: its six training traces take a median of 32.8 s for EXTRA (446× native, 0.74 of a full trace at the test config), and for INTER, whose training includes the larger MEDIUM config, 10.9 times the full trace it is meant to avoid.
- **Static α is dominated** by the static footprint: it needs the same interpreter run plus a sampled run, and is less accurate (1.19% against 0.09%). Its value is where the footprint

itself is not the target, e.g. in correcting sampled bins, or where a sampled run is taken anyway.

- **Borrowing by measured z, and the oracle, cost at least a full trace** of C, which already gives the exact footprint; they are upper bounds for the descriptor, not methods.
- **Enumerating every iteration** (the interpreter before symbolic counting) costs 7.79× more at MEDIUM (median 1.9 s, 25× native), of the order of Pin's instrumentation alone (41×); it grows with the program's references, symbolic counting with the touched range (Figure 3).

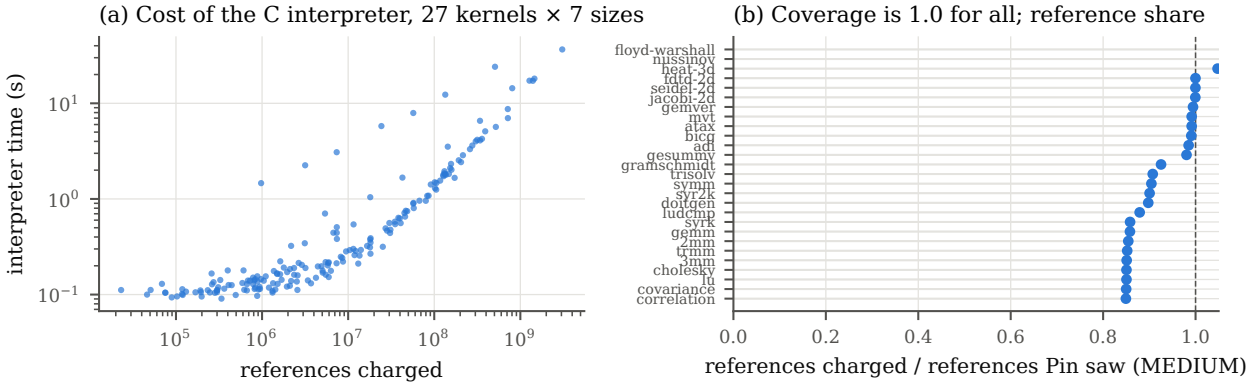


Figure 20: (a) Interpreter time per kernel and config, every iteration enumerated (symbolic counting: Figure 3). (b) References the interpreter charges as a share of those Pin records, at MEDIUM. Coverage is 1.0 for every kernel.

7.8 The gate on irregular programs

Table 9: Access idioms of PolyBench and of irregular programs.

Program	affine	indirect	pointer	data-bounded loops	while loops	gate
PolyBench (30 kernels, min / max)	0.986–1.000	0.000	0.014	0.00	0.00	in
miniVite	0.792	0.026	0.182	0.51	0.01	out
gap-bfs	0.722	0.108	0.170	0.40	0.12	out
gap-pr	0.683	0.086	0.231	0.56	0.07	out
gap-tc	0.204	0.038	0.758	0.17	0.73	out
darknet	0.793	0.008	0.095	0.21	0.01	out

All PolyBench kernels pass the gate; miniVite, GAP and darknet fail it (Table 9, Figure 21). On miniVite the call is right (Figure 22): its own model has 11.4% error at its largest input, the PolyBench model nearest by measured z (3mm) 64.5%, the best PolyBench model chosen after the fact 46.3% and the median one 80.0%. Its measured z is 4.3 from the nearest PolyBench kernel, where PolyBench kernels' own nearest-neighbour distances are at most 1.84, so the descriptor flags it as out of distribution as well.

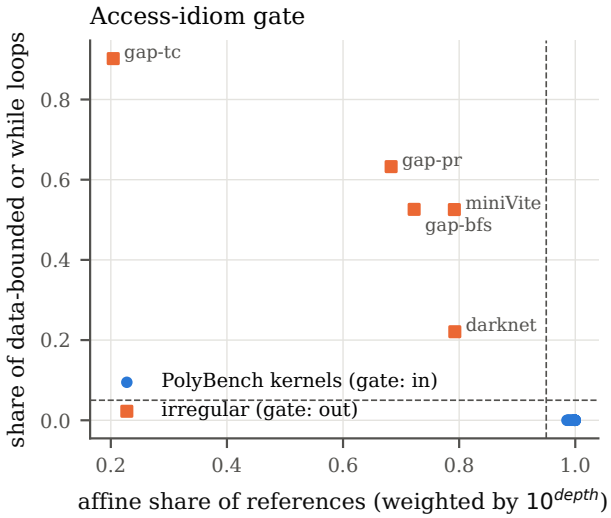


Figure 21: Affine share against the share of data-bounded or while loops. Dashed: the gate's thresholds.

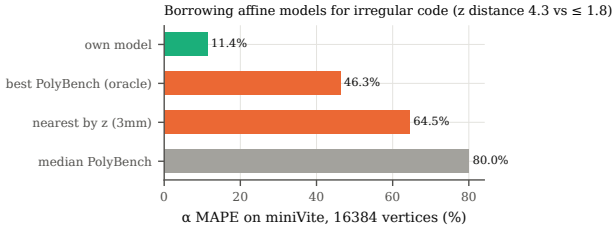


Figure 22: Errors on miniVite of its own model and of borrowed PolyBench models.

8 Irregular code I: a skeleton and the input's distribution

In an irregular program the access counts depend on the input: a vertex is read once per neighbour. Yet benchmark inputs are often *generated* by the program itself from a known distribution, as in GAP. The spectrum then depends only on the generator's distribution, which the source states. We first test this idea with hand-written *skeletons* (`static/skeleton.py`, `static/gap.py`, `static/gap_kernels.py`, about 900 lines), and then replace them with interpreters (Section 9).

8.1 The input distribution

GAP generates its graphs inside the program: `MakeUniformEL` draws both endpoints of every edge uniformly with a Mersenne Twister [21] (`std::mt19937`, reseeded every 2^{18} edges); `MakeRMatEL` draws, per edge and per depth, one 32-bit number that selects a quadrant with probabilities $A = 0.57$, $B = C = 0.19$, then permutes the vertex IDs with `std::shuffle`. The skeleton draws its graph from the same distribution with numpy (not GAP's actual edges). Figure 23 shows the resulting degree distributions: Poisson-like for uniform graphs, heavy-tailed for Kronecker graphs.

8.2 The skeleton

A skeleton is a numpy replay of the program's memory behaviour on the sampled input. A `Process` object provides `new(bytes)`, `delete(block)` and `touch(block, index, elem_size, refs)`, which adds reference counts per 4-byte unit. For GAP it replays:

- **The builder**: edge-list generation, `FindMaxNodeID` (an extra read of the edge list because `num_nodes_` starts at -1), degree counting with atomic increments, a prefix sum, CSR construction, `SquishCSR` (sort, unique and remove of every neighbour list), and symmetrisation.
- **The kernels**: pr's 20 iterations; bfs's top-down and bottom-up steps and direction switches on the sampled graph; cc's Afforest (Link, Compress, 1,024-sample guess of the largest component); sssp's Δ-stepping with per-bin vectors on the weighted builder; tc's `WorthRelabelling`, `RelabelByDegree` and `OrderedCount`; bc's Brandes from one source; prc's convergence loop, with its iteration count obtained by running the `float32` update on the sampled graph. Sequential parts run in numba [16].
- **Library costs**, measured outside GAP: `std::sort` references per element along a curve measured under Pin for lists of 2–16,384 integers (Table 10); `lock xadd` as a read and a write; the Mersenne Twister's 624 state words of 8 bytes, each referenced 2 times per seeding and $4 \cdot \text{draws}/624$ times per batch of draws.

Table 10: References per element of `std::sort` on d random 4-byte integers, measured under Pin (g++ 8.5, `-O3`); interpolated in $\log_2 d$.

d	2	4	8	16	32	64	256	1024	4096	16384
refs/element	3.25	5.55	8.22	12.61	13.52	15.41	19.08	23.09	26.60	30.77

8.3 The glibc heap model

Because the footprint counts an address once however many blocks occupy it, where blocks are placed matters. The model follows glibc 2.28 [11]: a request of n bytes needs a chunk of $\max(32, \lceil (n+8)/16 \rceil \cdot 16)$ bytes; it is served best-fit from free heap chunks, else from the top

chunk; only when the top chunk is too small does the size decide between a mapping (at or above the mmap threshold) and growing the heap by the request plus a 128 KiB pad, rounded to pages. Freed chunks coalesce, and into the top chunk when adjacent; a free that leaves more than the trim threshold in the top chunk returns memory to the OS down to the pad. Freeing a mapping raises the mmap threshold to its size (up to 32 MiB) and the trim threshold to twice that. Under Pin, holes left by `munmap` are not reused (the tool's own mappings take them), which the model reproduces. The heap's starting state, 59,328 free bytes in the top chunk at GAP's first large allocation (libstdc++'s 72,704-byte emergency pool and the command line are already there), was measured once with `mallinfo()` from an `LD_PRELOAD` shim. At scale 14, a 131 KB array lands on the heap or in a mapping by a margin of about 2 KB; this model brought the footprint error there from 1.8% to under 0.4%.

8.4 Calibration and blind tests

The uniform-graph pr and bfs results are *not* blind: two corrections were found by comparing against their traces (generated graphs are always symmetrised, `symmetrize_ = true`; the generator state uses 8-byte words). Three further corrections were measured outside GAP (the sort curve, the cost of `lock xadd` and glibc's placement). Everything after that was tested **blind**: the model was committed before the workloads it predicts were traced, and not changed after scoring. The blind tests were: Kronecker graphs for pr and bfs (only the generator was added), the five further kernels cc, sssp, tc, bc and prc on both graph families, and pr and bfs with 4 OpenMP threads (per-thread generators and arenas, per-thread queue buffers). One disclosure: for prc, the native iteration counts were printed before the freeze commit; the replay was not changed.

8.5 Results

Table 11 and Figure 25 summarise the 36 held-out tests. The skeleton beats each workload's own model in 28 of them; borrowed PolyBench models are 16–98% off. The footprint is predicted within 0.0–2.9% at every scale (Figure 24). The blind Kronecker test passes with α within 1.2–2.2%, more accurate than on uniform graphs, presumably because the skewed degrees move more of the footprint into hub vertices and long neighbour lists, which the replay gets right (an untested explanation). sssp (7.3–11.3%) and tc (9.0–11.2%) are worse than their own models; their coarsest parts are sssp's per-bin vectors and tc's merge loops, whose cost depends on how `-O3` keeps the iterator in a register. Threads did not make prediction harder (0.8–4.5%): every array access is the same as in the serial run. Figure 26 shows the error at every scale; it falls with scale for pr, bfs and cc, as the runtime baseline becomes a smaller part of the footprint.

8.6 Irregular C programs run by the interpreter

The interpreter of Section 4.7 was tested on two C programs written for the purpose, `csr_pr.c` and `csr_bfs.c`: both build a random CSR graph from `rand()`, then run PageRank or a queue-driven BFS. They were built `-O0` and traced at scales 8–16 (pr) and 8–14 (bfs); the baseline was fitted on PolyBench. The blind run missed the C library's internal references per `rand()` call; adding the measured cost brought α to 1.3–6.0% (Table 12, Figure 27). That correction is post hoc. At their largest scales the interpreter costs 36 s for csr_pr (scale 16: 0.40 of a full trace, 10× a sampled run) and 190 s for csr_bfs (scale 14: 26× a full trace, 102× a sampled run): BFS's queue loop claims vertices through memory, so it runs one vertex at a time (Table 20).

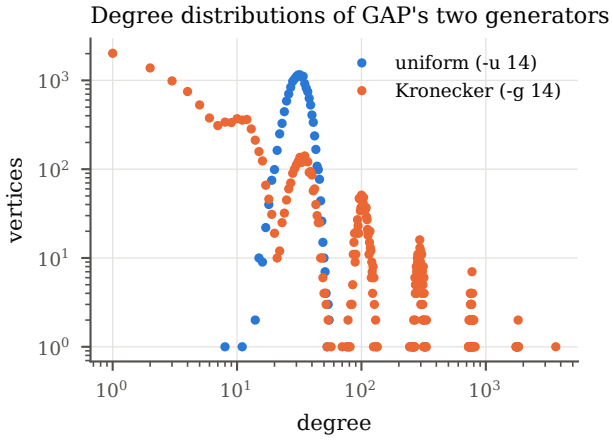


Figure 23: Degree distributions of the graphs drawn from GAP's two generators at scale 14.

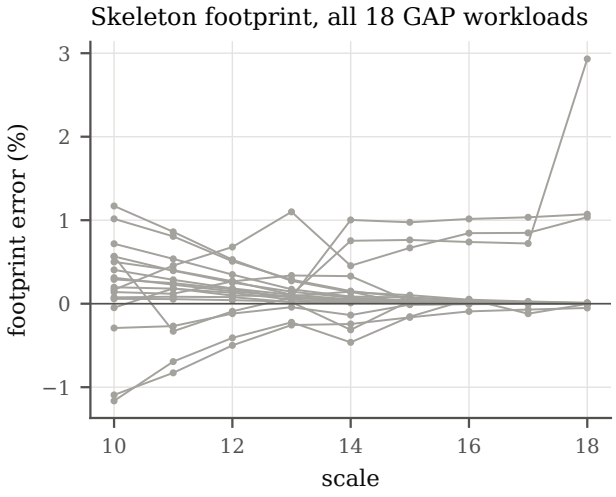


Figure 24: Skeleton footprint error at every scale, all GAP workloads.

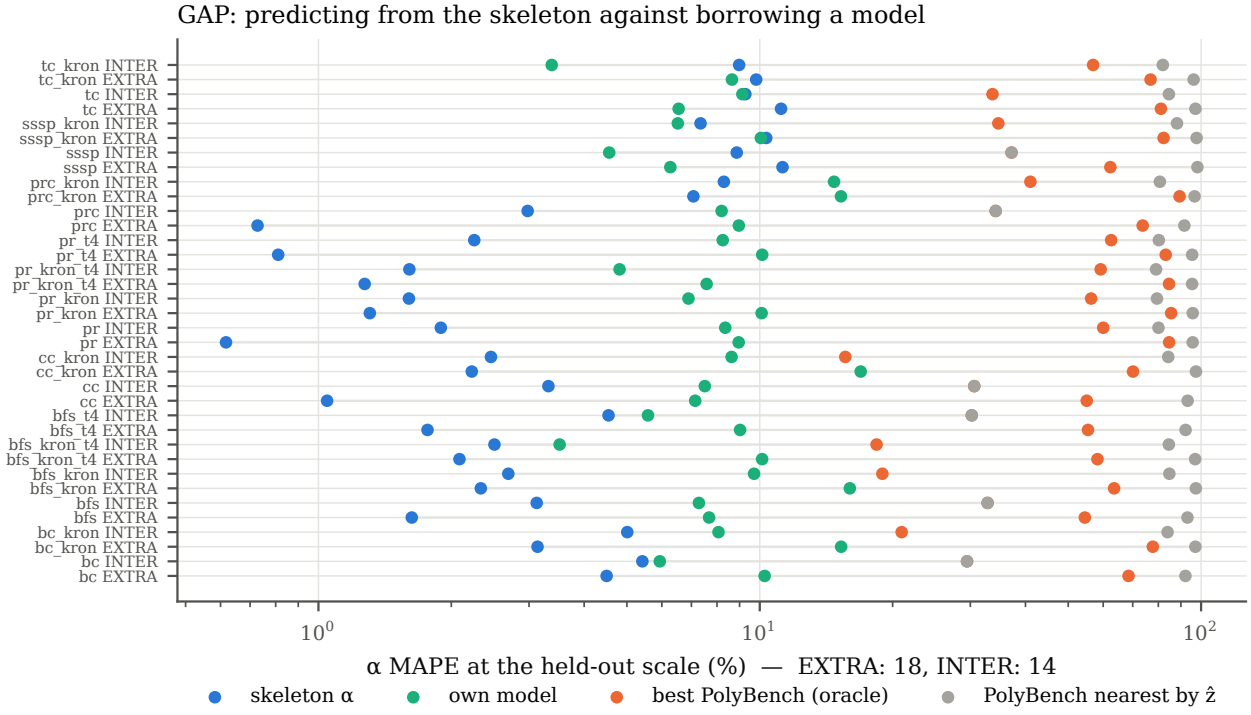


Figure 25: GAP: α error of the skeleton, the workload's own model and the borrowed PolyBench models, per workload and split.

Table 11: GAP skeleton: footprint error and α MAPE (%) at the held-out scale (EXTRA: 18, INTER: 14), against the workload's own model and borrowed PolyBench models (nearest by $\hat{z}$, uniform mean, oracle). The baseline is fitted on other GAP workloads.

Workload	split	skel. fp	skel. α	own	PB nearest	PB mean	PB oracle	blind
bc	EXTRA	0.00	4.5	10.3	92.2	92.2	68.5	yes
bc	INTER	0.14	5.4	5.9	29.5	85.3	29.5	
bc_kron	EXTRA	0.00	3.1	15.3	97.1	90.9	77.7	yes
bc_kron	INTER	0.33	5.0	8.1	83.9	82.3	21.0	
bfs	EXTRA	0.00	1.6	7.7	93.1	93.0	54.5	no
bfs	INTER	0.14	3.1	7.3	32.8	86.6	32.8	
bfs_kron	EXTRA	0.00	2.3	16.0	97.3	92.1	63.5	yes
bfs_kron	INTER	0.31	2.7	9.7	84.7	83.1	19.0	
bfs_kron_t4	EXTRA	0.00	2.1	10.1	97.0	90.9	58.2	yes
bfs_kron_t4	INTER	0.46	2.5	3.5	84.5	83.1	18.4	
bfs_t4	EXTRA	0.05	1.8	9.0	92.2	92.5	55.4	yes
bfs_t4	INTER	0.24	4.5	5.6	30.2	86.1	30.2	
cc	EXTRA	0.00	1.0	7.1	93.2	93.1	55.1	yes
cc	INTER	0.07	3.3	7.5	30.6	86.2	30.6	
cc_kron	EXTRA	0.01	2.2	16.9	97.4	92.1	70.1	yes
cc_kron	INTER	0.09	2.5	8.6	84.2	82.7	15.6	
pr	EXTRA	0.00	0.6	9.0	95.7	85.2	84.6	no
pr	INTER	0.04	1.9	8.4	80.1	78.3	60.0	
pr_kron	EXTRA	0.00	1.3	10.1	95.7	85.7	85.5	yes
pr_kron	INTER	0.03	1.6	6.9	79.4	77.3	56.3	
pr_kron_t4	EXTRA	0.01	1.3	7.6	95.4	84.8	84.6	yes
pr_kron_t4	INTER	0.14	1.6	4.8	79.1	77.0	59.2	
pr_t4	EXTRA	0.01	0.8	10.1	95.4	84.1	83.1	yes
pr_t4	INTER	0.15	2.3	8.2	80.2	78.4	62.5	
prc	EXTRA	0.00	0.7	9.0	91.6	91.6	73.7	partly
prc	INTER	0.01	3.0	8.2	34.2	84.1	34.2	
prc_kron	EXTRA	0.00	7.1	15.3	96.7	89.4	89.4	partly
prc_kron	INTER	0.02	8.3	14.7	80.6	78.9	41.0	
sssp	EXTRA	1.04	11.3	6.3	98.1	94.0	62.3	yes
sssp	INTER	0.45	8.9	4.6	37.2	89.2	37.2	
sssp_kron	EXTRA	1.07	10.3	10.1	97.7	92.2	82.2	yes
sssp_kron	INTER	1.00	7.3	6.5	88.2	86.9	34.7	
tc	EXTRA	0.00	11.2	6.5	97.1	91.1	81.1	yes
tc	INTER	0.05	9.3	9.1	84.5	83.1	33.7	
tc_kron	EXTRA	2.93	9.8	8.7	96.2	78.3	76.8	yes
tc_kron	INTER	0.75	9.0	3.4	81.9	79.9	56.9	

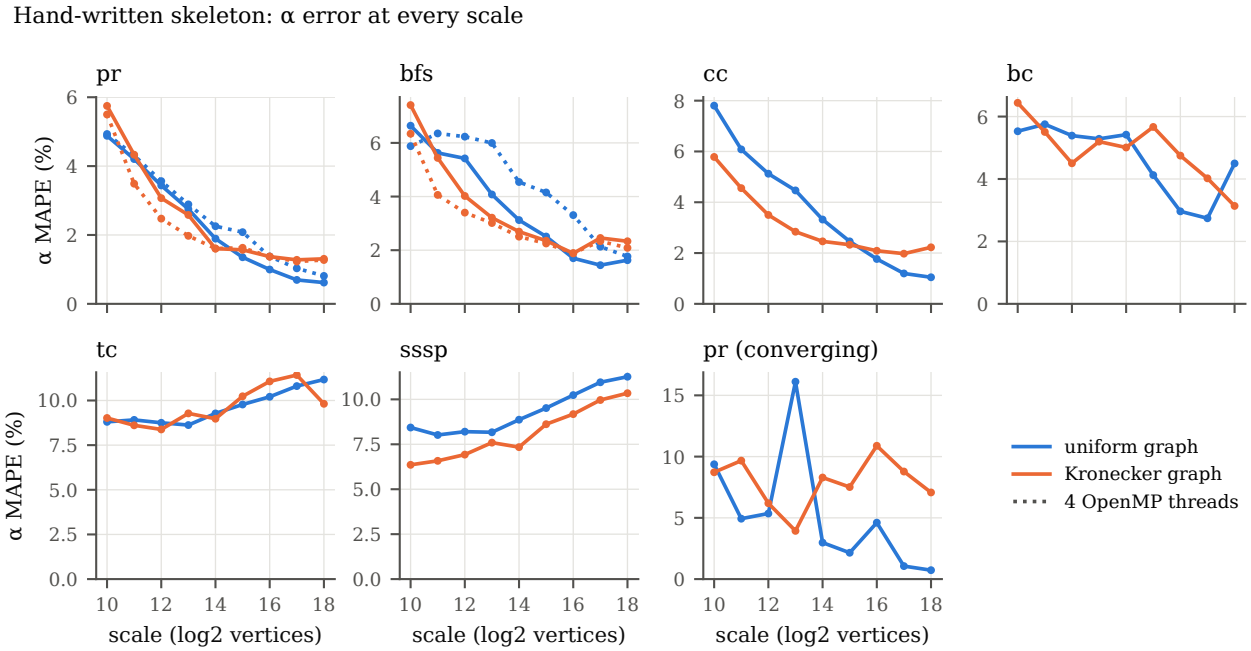


Figure 26: Skeleton α MAPE at every scale, for the seven kernels, both graph families and (pr, bfs) 4 threads.

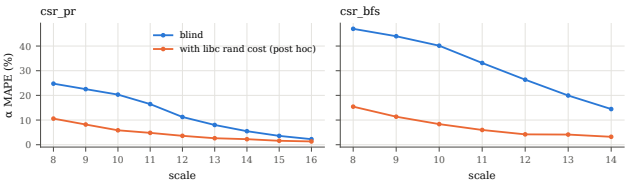


Figure 27: α MAPE per scale of the interpreter on two irregular C programs, blind and with the libc `rand()` cost.

Program	split	run	footprint	static α	own model
csr_bfs	EXTRA	blind	0.01	14.5	20.3
csr_bfs	EXTRA	post hoc	0.02	3.2	20.3
csr_bfs	INTER	blind	0.11	33.1	14.2
csr_bfs	INTER	post hoc	0.15	6.0	14.2
csr_pr	EXTRA	blind	0.00	2.2	12.4
csr_pr	EXTRA	post hoc	0.00	1.3	12.4
csr_pr	INTER	blind	0.02	11.2	11.9
csr_pr	INTER	post hoc	0.05	3.6	11.9

Table 12: Footprint and α MAPE (%) on the two C programs.

9 Irregular code II: interpreting GAP's C++ source

The skeletons work but are written by hand. The C++ interpreter (`static/interp_cpp.py`, about 2,000 lines, and `static/containers.py`) removes that step: it runs GAP's unmodified source, from `main()`, through the command-line parser, the generator and builder, the kernel and the destructors.

9.1 Object model

- **Objects and members.** Fields sit at the offsets libclang reports; `this` is a pointer; member calls, operators and lambdas' `operator()` are dispatched on the referenced declaration. libclang's `get_arguments()` omits the object of an operator call; whether it is included is decided by comparing the argument count with the callee's parameter count.
- **Construction and destruction.** Constructors run member initialisers, default member initialisers and delegation; an implicitly defined `operator=` (no parameter declarations in the AST) is executed as a member-wise copy. Objects returned by value are constructed in storage the caller provides (sret); a returned local is not destroyed. Destructors run at scope exit (libclang does not show those calls).
- **References** hold the address they bind to; a call that returns a reference is an lvalue. A scalar bound to a reference (an out-parameter such as `&index`) is materialised in memory.
- **Pointers in memory** are remembered as (object, offset) per 4-byte unit, so arrays of pointers such as GAP's CSR index resolve.
- **Templates.** libclang gives instantiated function bodies with concrete types; for class-template patterns the interpreter resolves fields on the concrete type.
- **Function values.** A function passed to a template parameter (tc passes `Hybrid` to `BenchmarkKernel`) evaluates to a function value, and a call through a variable holding one invokes its definition.

9.2 Values

- **Floating-point values** (`Real`) are tracked through literals, casts, arithmetic, comparisons and the C math functions (`pow`, `sqrt`, `cbrt`, `exp`, `log`, `fabs`, `floor`, ...), in scalars and in memory: every memory object keeps a float per 4-byte unit beside its integers (NaN when unknown; an integer store forgets it, and a unit whose integer value is 0 reads as 0.0). So a decision such as tc's `sample_average / 1.3 > sample_median`, or a problem parameter stored in a struct, resolves. A floating-point compound update of memory (`f[node[e]] += x`) is applied in iteration order with `np.add.at`, so repeated indices accumulate as in a sequential loop. A constant floating-point expression converted to an integer is folded by clang (`const uint32_t A = 0.57*max`).
- **Per-point values at one address.** A local array in the body of a batched loop has one address for all iterations (which is right for counting), but each iteration writes its own values. When a batch stores an array of values to a single address, the object keeps them per point, and a load from that address by a batch of the same size returns each point's value.
- **Partial values.** A variable assigned in the branches of an `if`/`else` that different batch points take is known per point once every point has been assigned (`Partial`); before, it became unknown. (Added after the blind test, Section 10.)

9.3 Counting as `-O3` does

GAP is traced at `-O3`, where most scalars never touch memory. The interpreter therefore:

- keeps scalar locals and objects of at most 64 bytes (an `Edge`, a `Neighborhood`, a `pvector`'s three pointers) in a per-batch-point register file whose accesses are not charged; taking such an object's address or binding it to a reference moves it to memory;
- eliminates redundant loads per batch point: a point's load of a location it (or the point it derives from in an enclosing batch) already accessed through the same base pointer, with no store to that object since, is free. Every batch records, per (object, access size, base-pointer provenance), the offset each point loaded, and links to the batch it was derived from: a subset of its points, the iterations of a vectorised loop, or one iteration of a loop run one at a time. Loads before a loop are therefore hoisted out of it (loop-invariant code motion), an iteration does not reuse what the previous one loaded, and the latest value a `while` condition loaded is kept for after the loop (tc's `while (*it < w) it++;` followed by `w == *it`). Provenance is the object a pointer was loaded from (and, for a small object, the field), the same in every batch. A store forgets every entry of its object; a library call that may write memory forgets everything.

9.4 The standard library

The standard library is modelled, not interpreted (its declarations are recognised by `is_in_system_header`):

- `std::vector`: three pointers (start, finish, end of storage) as in libstdc++; buffers come from one arena object so that a batched loop over many vectors works on one object; `push_back` grows by doubling (copy, release); `resize` value-initialises new elements, and nested vectors start as empty vectors.
- `std::unordered_map`: nodes (key, value) of 16 bytes in the arena; bucket arrays are not modelled.
- `std::sort` (field-wise lexicographic order, `std::greater` descending; cost from Table 10), `unique`, `remove`, `copy` (whole elements move), `fill`, `shuffle` (libstdc++ 8's two-positions-per-draw variant), `swap`, `max_element`, `min`, `max`, `numeric_limits`, `std::pair` and `make_pair`;
- `std::mt19937` and `uniform_int_distribution`: values (a generator per call site, seeded as in the source) and the state-word costs of Section 8;
- GAP's atomic helpers `fetch_and_add` and `compare_and_swap` (and the GCC builtins): applied in iteration order across batch points, so the first point to reach a location wins, as in a sequential run.

Streams and strings are ignored; command-line accessors come from a table of stubs (scale, degree, graph family, number of trials).

9.5 Control flow

- `break` and `continue` are per-loop flags; a loop whose body may break runs one iteration at a time for the batch points still in it (`_loop_with_break`, `_range_with_break`). A `break` under a condition on unknown (floating-point) data is ignored, so the loop runs to its bound; this is exact for pr with `-t 0` and an upper bound for converging loops.

- `range-for` runs through `begin()/end()` as a batch, sequentially if it carries values through memory.
- When only some batch points run a statement (after a partial `break`), the statement runs on that subset; variables it declares are merged back to the full batch.
- **Reductions and scans.** A loop whose carried scalar is only summed, or set to `min`/`max` of itself, runs as one batch and combines the iterations per enclosing point. A loop whose carried integer or pointer is only changed by increments that do not read it (`v += x`, `v++`, `*p++ = x`), anywhere in the body including branches and nested loops, and that no condition reads, is a *scan*: a first pass that charges and stores nothing (and restores every variable) gives each iteration's increment, and each iteration then starts from v_0 plus the increments of the earlier iterations of its enclosing point. GAP's prefix sum and HPCCG's row-by-row matrix generation run this way instead of one iteration at a time.
- **Loop conditions.** In a conjunction (`k < max_iter && normr > tolerance`), a part that compares floating-point values and is unknown is taken as true when the other parts are known, so the loop runs to its integer bound, consistent with the rule for `break`.

Programs of several source files are compiled as one unity file (headers without an include guard get `#pragma once`); the command line is given as strings (`argv`; `atoi`, `strtol`, `std::stoi`, `strcmp` and `strlen` read them); class-type fields without an initialiser are default-constructed (`std::vector` members start empty).

9.6 Bugs found by comparing against the hand skeleton

Comparing the interpreter's heap blocks, one by one, with the skeleton's exposed interpreter errors that a footprint check alone would have hidden. Table 13 lists those that changed results.

9.7 Results

Table 14 and Figure 28 compare the interpreter with the hand skeleton against Pin, with the same runtime baselines. All six kernels run on both graph families with coverage of 1.0 (sssp at scale 12: 0.9999). The footprint is within 2.94% everywhere (Figure 30; largest on tc and sssp with Kronecker graphs), and the total reference count within 3% of the skeleton's for every kernel (Table 14, last column). All six kernels were computed at all nine scales. For pr, bfs and tc, α is within about a point of the hand skeleton at every scale. At scales 13–18 of pr and bfs, which were never used while the interpreter was developed, α is within 0.66–3.97%, against 0.62–4.08% for the hand skeleton, and the footprint within 0.44% (Table 15). The per-point load cache closed the gap the per-batch cache left on tc (15.6% to 9.4% at scale 10; skeleton 8.8%) and bc (11.7% to 6.6%; 5.5%): the per-block comparison (Figure 32) matches the skeleton within 0.3% of tc's references and 0.9% of bfs's.

Three kernels changed with the fixes made while preparing the second blind test (Section 11, the last rows of Table 13). Before them, cc and bc drifted away from the skeleton on uniform graphs as the scale grew (cc reached 8.3% at scale 18 where the skeleton has 1.1%, bc 10–11% at scales 15–17 against 3–4%), and sssp looked better than the skeleton (7.2% against 9.3% mean). The per-block comparison found the causes: Afforest's largest-component skip in cc never fired, because the interpreter could not dereference an `unordered_map` iterator and the most frequent component stayed unknown, so every vertex traversed its whole neighbourhood; bc's successor bitmap retried its compare-and-swap about $k^2/2$ times per word when k batch points set bits of one word; and sssp's `std::copy` of a bin into its frontier stored nothing, so the algorithm stopped after its first

Table 13: Interpreter defects found on GAP and their fixes.

Symptom	Cause and fix
sssp ran forever	`vector<vector<int>>::operator[]` returns the typedef `reference`; it was not recognised as a reference, so each bin was copied into a temporary and `empty()` was unknown. Reference types are now canonicalised.
Kronecker graphs not generated (coverage 0.90, footprint 40% of truth)	`0.57*max` evaluated to unknown; constant float-to-integer conversions are now folded.
tc's kernel never ran	a function passed as a template argument was not callable; function values added.
tc never relabelled	`WorthRelabelling` decides on `double` locals; floating-point scalars added; then `std::pair` construction, assignment and sort.
tc's iterator lost	a variable declared while only some batch points ran was dropped on merging.
pr and bfs wrong above scale 16	GAP's prefix sum runs one iteration at a time over all vertices; it hit the 100,000-iteration guard at 2^{17} vertices. Guard raised to 2^{24}.
packed `bool` fields clobbered	values were kept per 4-byte unit only; per-byte values added.
sssp weights lost	`copy`/`unique` moved only the first 4 bytes of 8-byte elements.
`break` in pr's convergence test stopped the loop	a break under an unknown condition is now ignored.
load elimination too aggressive (10–12% error)	loads through different base pointers were merged; provenance added to the key.
tc and bc 6–9 points worse than the skeleton	the load cache was per batch; made per point with parent batches (above).
nested sums multiplied by the outer trip count	an inner reduction was combined over all points instead of per enclosing point.
pointer increments lost	`p++` was summed as an integer reduction; pointers advance as scans.
two out-parameters shared a slot	a scalar whose address is taken in an inner block was given scoped scratch space; it now gets its own region.
Found while preparing the second blind test (Section 11):	
cc drifted from the skeleton with scale	`unordered_map` iterators were not modelled, so Afforest's most frequent component was unknown and its largest-component skip never fired; iterators now hold a node pointer. Its two neighbour rounds also ran as one batch; loops of at most 16 rounds with inner loops now run in order.
bc drifted from the skeleton with scale	a compare-and-swap retry loop over a fresh read of the same word failed for all but one of the batch points setting bits of that word; a retry that updates bits now succeeds, merging the bits.
sssp better than the skeleton	`vector` iterators were copied as objects, so `std::copy` of a bin into the frontier stored nothing and the algorithm stopped after one round; iterators now hold their pointer. Then each batched `push_back` charged the whole batch's weight per vector; now each point's.

Table 14: GAP from its C++ source: largest footprint error and mean α MAPE over the scales computed, for the hand skeleton and the interpreter, and the ratio of their total reference counts (median over scales).

Kernel	graph	scales	max |footprint error| (%)		mean α MAPE (%)		interp./hand
			hand	interp.	hand	interp.	refs
pr	uniform	10–18	0.31	0.32	2.3	2.3	1.00
pr	Kronecker	10–18	0.29	0.30	2.5	3.0	1.00
bfs	uniform	10–18	0.29	0.08	3.6	3.5	1.00
bfs	Kronecker	10–18	0.31	0.50	3.5	4.3	1.01
cc	uniform	10–18	0.57	0.58	3.7	3.9	1.01
cc	Kronecker	10–18	0.72	0.84	3.1	4.0	1.01
bc	uniform	10–18	0.18	1.69	4.6	4.4	0.97
bc	Kronecker	10–18	0.34	1.48	4.9	3.9	0.99
tc	uniform	10–18	0.41	0.42	9.6	10.4	1.00
tc	Kronecker	10–18	2.93	2.94	9.6	10.0	1.01
sssp	uniform	10–18	1.10	2.79	9.3	8.9	1.03
sssp	Kronecker	10–18	1.07	2.91	8.1	8.8	1.03

round and its lower error was an accident. Now cc is within 1.7 points of the skeleton at every scale (2.0% against 1.1% at scale 18; 3.9% against 3.7% mean on uniform graphs), bc is better than the skeleton on average (4.4% against 4.6% mean on uniform graphs, 3.9% against 4.9% on Kronecker graphs; worse at scale 11, 8.9% against 5.8%), and sssp, now run to the end, tracks the skeleton within 1.6 points (8.9% against 9.3% mean on uniform graphs, 8.8% against 8.1% on Kronecker graphs), with its footprint within 2.9%.

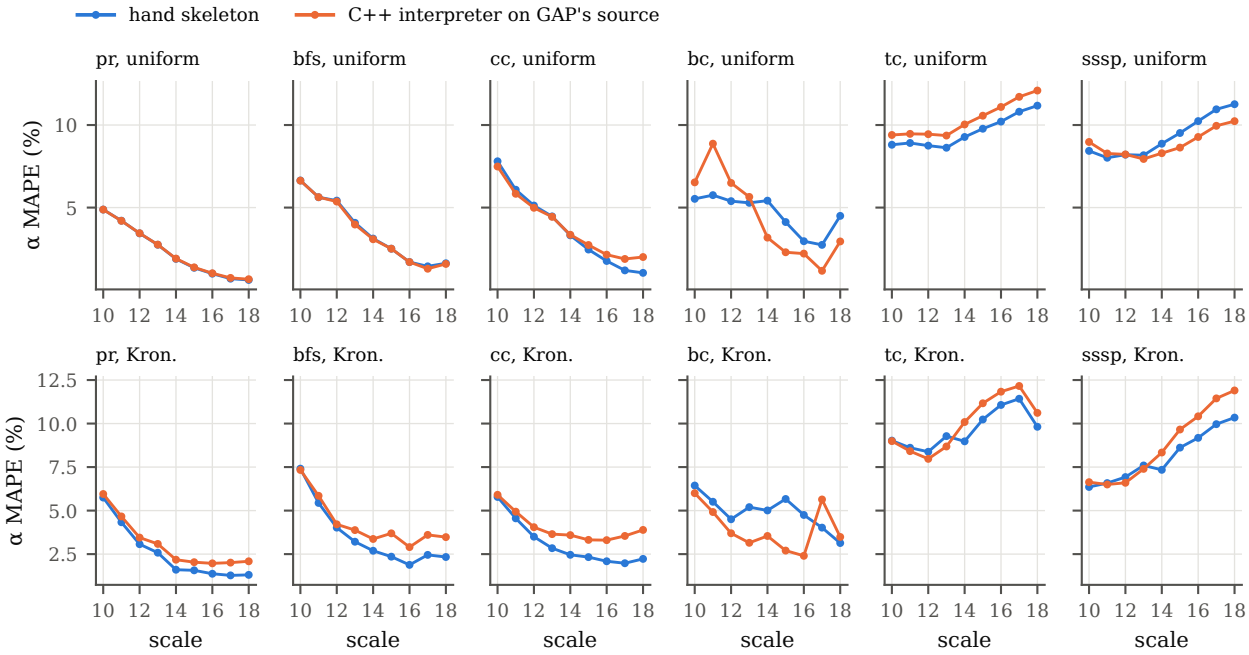


Figure 28: α MAPE per scale: hand skeleton against the C++ interpreter on GAP's source, six kernels, uniform (top) and Kronecker (bottom) graphs.

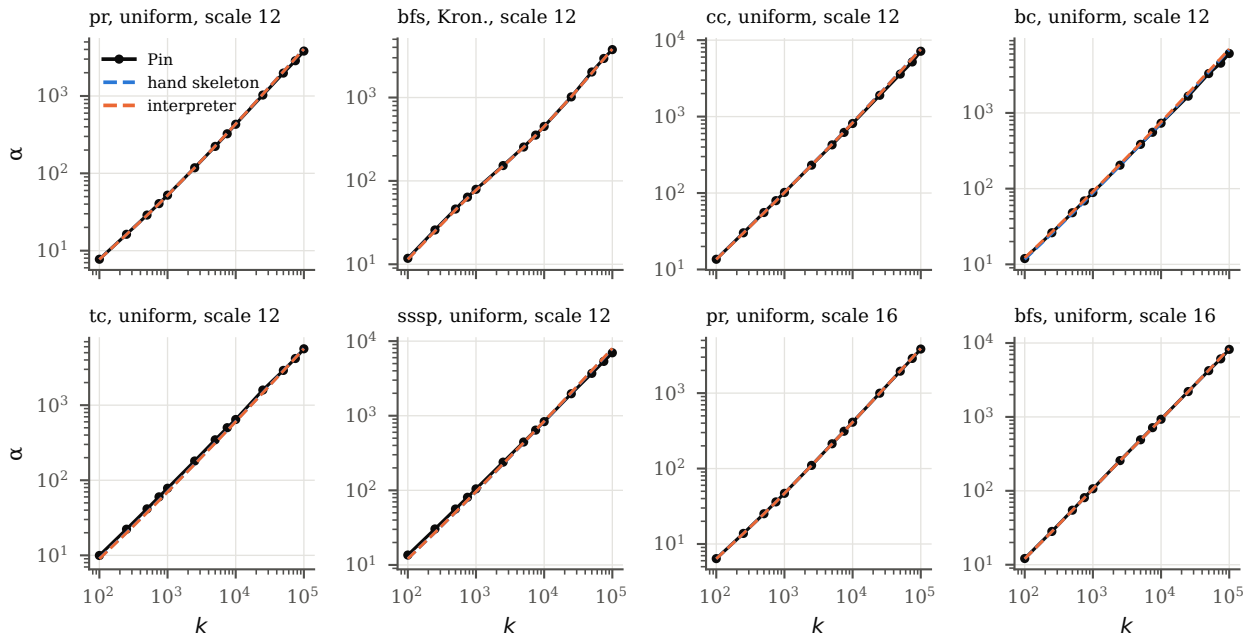


Figure 29: α against the interval: Pin, the hand skeleton and the interpreter, for eight workload–scale pairs.

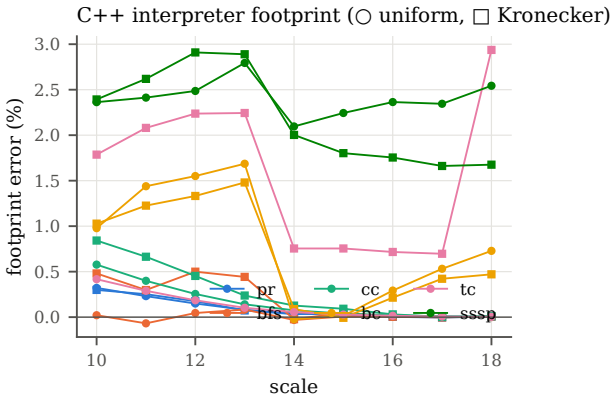


Figure 30: Footprint error of the C++ interpreter per scale.

Scale	pr, uniform		pr, Kron.		bfs, uniform		bfs, Kron.	
	hand	interp.	hand	interp.	hand	interp.	hand	interp.
10	4.9	4.9	5.7	6.0	6.6	6.6	7.4	7.3
11	4.2	4.2	4.3	4.7	5.6	5.6	5.4	5.9
12	3.4	3.4	3.1	3.4	5.4	5.4	4.0	4.2
13	2.8	2.8	2.6	3.1	4.1	4.0	3.2	3.9
14	1.9	1.9	1.6	2.2	3.1	3.1	2.7	3.4
15	1.4	1.4	1.6	2.0	2.5	2.5	2.4	3.7
16	1.0	1.0	1.4	2.0	1.7	1.7	1.9	2.9
17	0.7	0.7	1.3	2.0	1.4	1.3	2.5	3.6
18	0.6	0.7	1.3	2.1	1.6	1.6	2.3	3.5

Table 15: α MAPE (%) per scale, pr and bfs.

9.8 Cost

The first version of the interpreter took about 13 minutes at scale 16 and 67–78 minutes at scale 18, almost all of it in GAP's prefix sum, which ran one vertex at a time. Scans, cached libclang answers (child lists, kinds and canonical types are cached on the cursor and type objects, declaration keys on the declaration) and counting over the touched range only brought pr and bfs to about 20 s at scale 16. The interpreter as frozen for the second blind test (Section 11) is slower again on some workloads, because it is more faithful: floating-point scalars read before they are written now count as carried (needed for LULESH), so pr's pull loop, a floating-point sum over each vertex's in-neighbours, runs one neighbour position at a time for all vertices, and on Kronecker graphs, whose largest degree is in the thousands, pr_kron takes 1,972 s at scale 18; cc, bc and sssp now run their algorithms to the end (the iterator and compare-and-swap fixes listed in Section 11). All

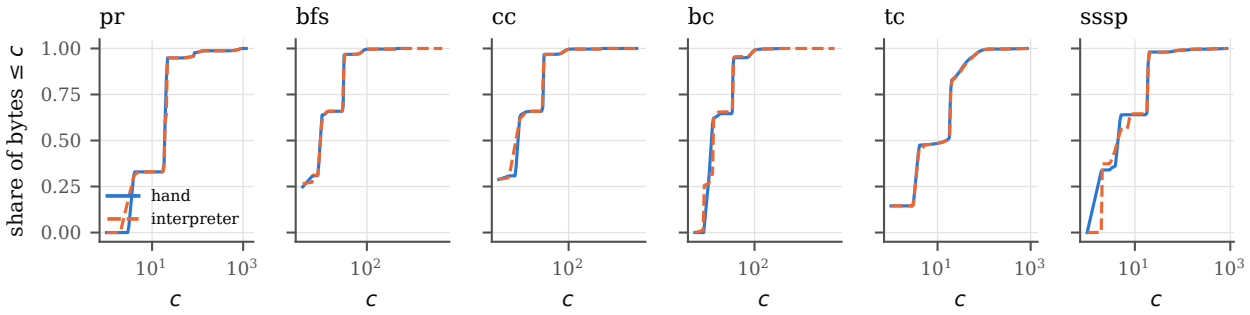


Figure 31: Cumulative spectra at scale 12 (uniform): hand skeleton against the interpreter.

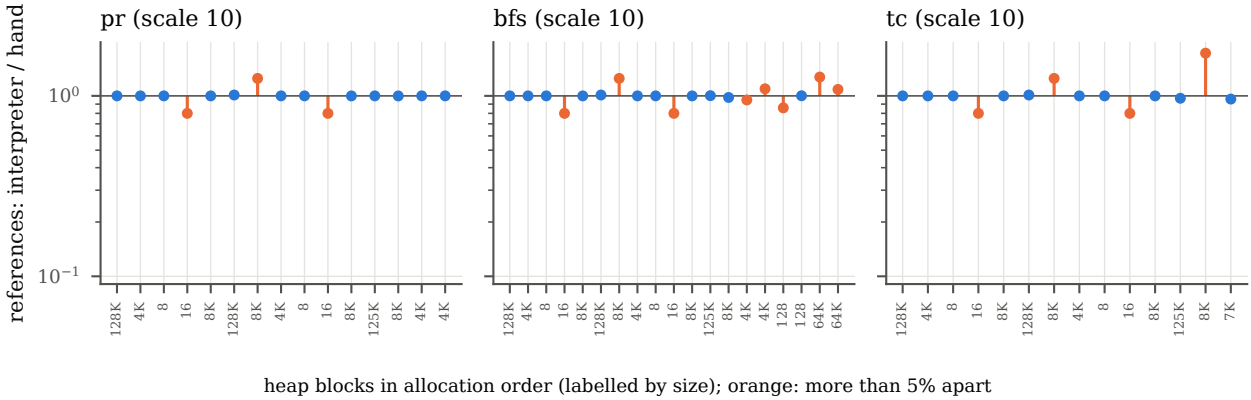


Figure 32: References to each heap block, interpreter over hand skeleton, at scale 10.

108 runs took 3.79 CPU-hours, recorded with 4–16 runs at a time (Table 16, Figure 33); 16 at a time ran the 62 GB node out of memory (one run at scale 18 had reached 9.5 GB), and the 23 lost runs were repeated with 4 at a time. Run alone, the scale-16 runs take a median of 58 s, half the time recorded in parallel (Table 22).

Against the alternatives at scale 16 (medians over the 12 workloads, Table 20): the native run takes 0.30 s, Pin's sampled run 5.0 s (17×), a full trace 90 s (266×), the hand skeleton 15 s and the interpreter 58 s alone (176× native, 0.68 of a full trace). At scale 18 the sampled run takes 12 s, the skeleton 67 s and the interpreter 257 s (recorded in parallel). So on GAP the interpreter costs about as much as a full trace and ten times a sampled run; the hand skeleton is cheaper to run but costs a person: about 1,075 lines of Python written for GAP's seven kernels (`gap.py`, `gap_kernels.py`, `skeleton.py`), against about 4,900 lines for the two interpreters, which contain no code for any particular program.

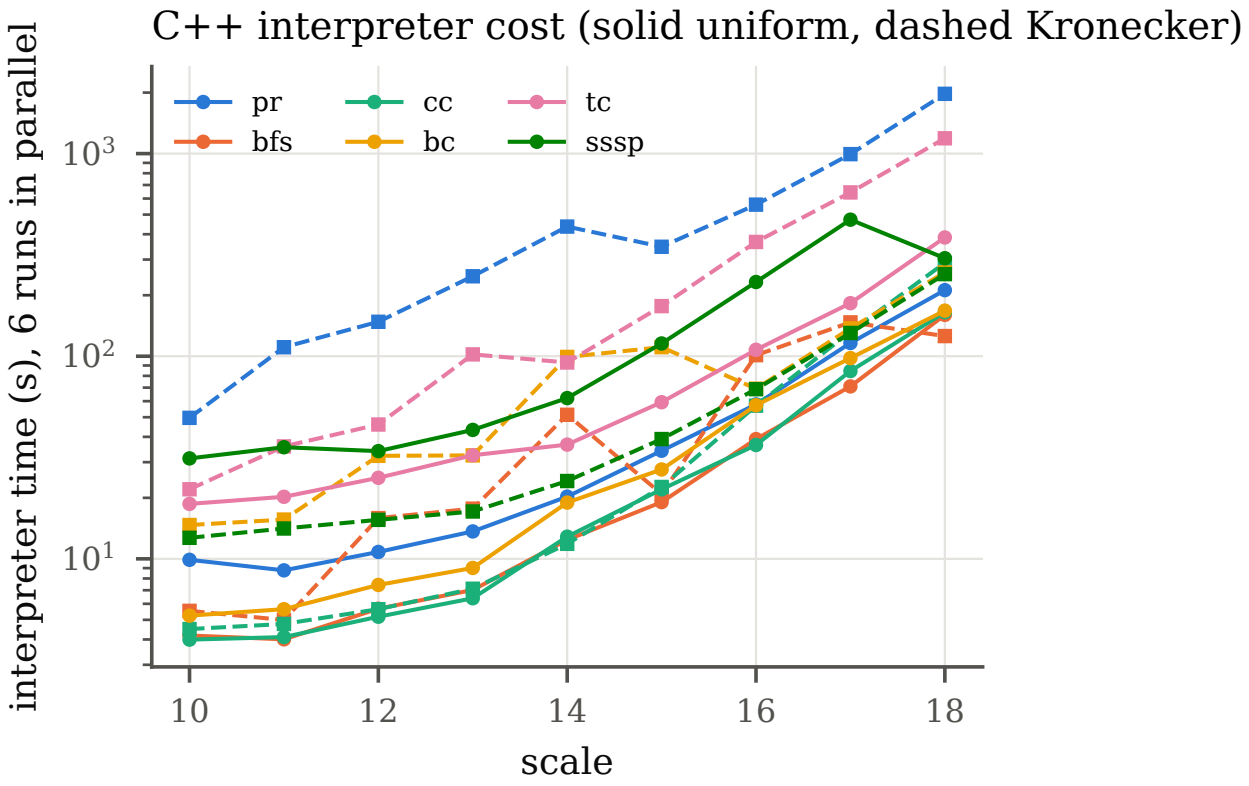


Figure 33: Wall time of the C++ interpreter per kernel, graph and scale.

Table 16: C++ interpreter seconds per kernel and scale as frozen for the second blind test (u: uniform, K: Kronecker; recorded with 4–16 runs at a time, so up to about twice the time of a run alone, Table 22).

Scale	pr u	pr K	bfs u	bfs K	cc u	cc K	bc u	bc K	tc u	tc K	sssp u	sssp K
10	10	50	4	6	4	5	5	15	19	22	31	13
11	9	111	4	5	4	5	6	16	20	36	36	14
12	11	148	6	16	5	6	7	32	25	46	34	16
13	14	248	7	18	6	7	9	32	32	102	43	17
14	20	436	12	51	13	12	19	99	37	93	62	24
15	34	347	19	21	22	23	28	111	59	177	116	39
16	58	560	39	101	36	57	57	69	108	366	232	69
17	116	994	71	147	85	132	98	137	183	643	472	130
18	212	1972	160	126	165	289	168	259	386	1190	305	254

10 A blind test on programs never seen in development

Everything so far was developed against the workloads it was scored on, or (GAP's large scales) against the same programs. To test whether the interpreter generalises to code it was not built for, we chose two programs that generate their input from their command line and were never used in development, ran them in the interpreter without any trace of them, committed the predictions, and only then traced them (`paper_generalization/blind/PROTOCOL.md`):

- **HPCCG** (Mantevo, `80dd2f1`): conjugate gradient on a 27-point stencil matrix generated for an n^3 grid; 150 CG iterations (tolerance 0); serial, `-O3`; $n = 10, 12, 14, 16, 20, 24, 28$.
- **LULESH 2.0** (LLNL, `3e01c40`): Lagrangian shock hydrodynamics on an s^3 hexahedral mesh, 11 material regions with load-imbalance repetitions, 20 cycles; serial, `-O3`; $s = 5, 8, 10, 12, 15, 18, 20$.

Protocol. Bringing the two programs up in the interpreter, before any trace, exposed generic gaps (floating-point values in memory, scans of pointers, command-line strings, unity builds, default construction of `std::vector` members, and four defects); they were fixed and the interpreter frozen at commit `20a8d1e`. The runtime baseline was fitted on the 12 serial GAP workloads (same compiler, libstdc++ and glibc), the heap started from GAP's measured state, and the predictions were committed at `2757a35`. Only then were the programs traced and scored; the predictions were not changed. The interpreter takes 11–48 s per HPCCG input and 39 s–5 minutes per LULESH input; coverage is 1.0 for HPCCG and 0.954 for LULESH.

Table 17: Blind test: predicted against traced footprint and α MAPE per config. The last column is LULESH after the post-hoc fix (partial values).

Program	config	truth (B)	predicted (B)	footprint error	α MAPE	post hoc α
HPCCG	10	813,170	830,342	+2.11%	4.36%	–
HPCCG	12	1,076,735	1,093,894	+1.59%	3.13%	–
HPCCG	14	1,450,320	1,467,462	+1.18%	2.29%	–
HPCCG	16	1,953,064	1,970,246	+0.88%	1.66%	–
HPCCG	20	3,423,048	3,440,262	+0.50%	1.17%	–
HPCCG	24	5,640,332	5,657,542	+0.31%	0.89%	–
HPCCG	28	8,768,247	8,776,286	+0.09%	0.53%	–
LULESH	5	582,903	567,210	-2.69%	14.68%	9.7
LULESH	8	868,203	848,114	-2.31%	5.22%	33.6
LULESH	10	1,248,064	1,196,326	-4.15%	4.80%	24.9
LULESH	12	1,760,008	1,712,782	-2.68%	4.74%	22.9
LULESH	15	4,217,052	4,170,114	-1.11%	28.09%	8.3
LULESH	18	6,868,580	6,840,642	-0.41%	36.33%	18.4
LULESH	20	9,234,875	9,190,430	-0.48%	42.86%	24.7

Results. (Table 17, Table 18, Figure 34.)

- **HPCCG passes clearly.** The footprint is within +0.09 to +2.1% and α within 0.5–4.4% at every grid size, improving with size as the runtime baseline becomes a smaller share. With the largest grid held out, the prediction from source is 0.53% off against 12.2% for the program's own model trained on its six other traced sizes; with the middle grid held out, 1.66% against 11.1%.

Table 18: Blind test against each program's own MemPrint model (trained on its other six configs), α MAPE (%) at the held-out config.

Program	split	held-out config	static α (blind)	own model	footprint
HPCCG	EXTRA	28	0.53	12.24	+0.09
HPCCG	INTER	16	1.66	11.05	+0.88
LULESH	EXTRA	20	42.86	7.87	-0.48
LULESH	INTER	12	4.74	17.03	-2.68

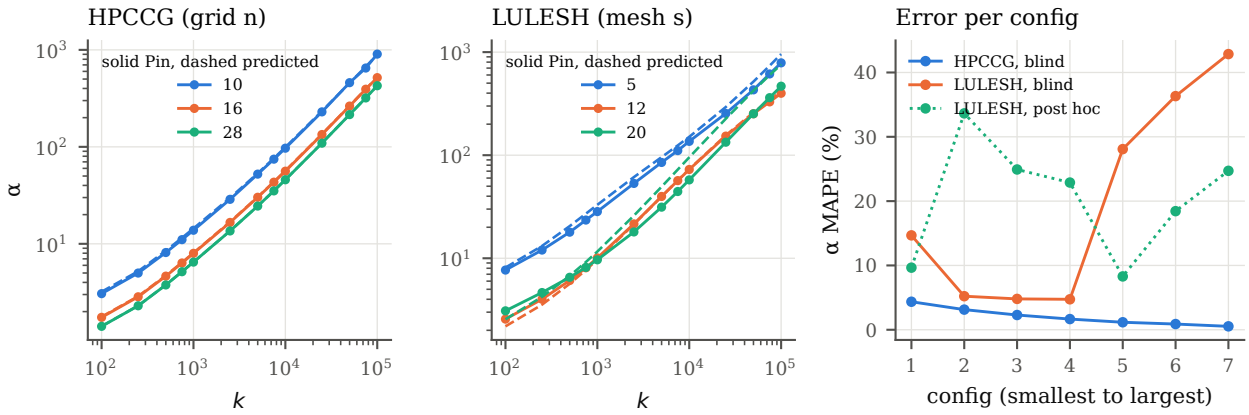


Figure 34: Blind test. Left and middle: α against the interval, traced (solid) and predicted before tracing (dashed), for three configs of each program. Right: α MAPE per config.

- **LULESH passes partly.** The footprint is within -4.2 to -0.4% at every mesh size, and α within 4.7–5.2% at $s = 8, 10, 12$ (INTER: 4.74% against the own model's 17.0%). At $s = 15, 18, 20$, however, α is 28–43% off (EXTRA: 42.9% against 7.9%): the predicted spectrum has too few heavily referenced bytes (at $k = 10^5$, α is 758 predicted and 467 measured at $s = 20$).

Post hoc. Looking for the cause after the traces existed, we found that a variable assigned in the branches of an `if/else` taken by different regions (LULESH's `rep`, how many times a region's equation of state is evaluated: 1, 2 or 20) stayed unknown, so those loops were charged one trip. Partial values fix this (GAP is unchanged by it), and LULESH's error at $s = 15$–20 falls to 8–25%, but at $s = 8$–12 it rises to 23–34% (Table 17, last column). The good middle sizes of the blind run therefore benefited from errors that cancelled, and LULESH's remaining error has more than one cause; the strongest candidate is that local coordinate arrays filled by a nested loop do not keep per-iteration values across the nested batch, so some floating-point time-step and cut-off decisions stay unknown (coverage 0.95–0.98). We report the blind numbers as the result.

Cost. At the largest configs (Table 20), the interpreter took 48 s for HPCCG ($n = 28$: 445× native, 0.27 of a full trace, 6.5× a sampled run) and 306 s for LULESH ($s = 20$: 1,376× native, 1.4 full traces, 35 sampled runs); these are the times recorded at the freeze.

11 A second blind test: NPB

Programs and protocol. Three kernels of the NAS Parallel Benchmarks in C++ (GMAP NPB-CPP, serial): IS (integer sort; classes S and W), CG (conjugate gradient on a random sparse matrix; S, W) and MG (multigrid on a 3-D grid; S, W, A), built `-O3` with NPB's own settings. None of NPB was used in development. Bringing them up found generic gaps, fixed before the freeze: global initialisers that are not constants (NPB allocates every array in one, `int *nx = malloc(...)` at namespace scope) now run before `main`; a scalar global whose address is taken (`randlc(&tran, amult)`) keeps its value in memory from then on; literal-only expressions are folded as `-O3` does; variable-length arrays are sized at run time; `fopen` returns `NULL`; loops that pass a scalar's address to a call, and loops of a few rounds, run one iteration at a time. Recomputing GAP's spectra for the baseline found further library gaps, fixed in the same freeze: `unordered_map` and `vector` iterators (Afforest's largest-component skip in cc, and `std::copy` of sssp's bins into its frontier, which had stored nothing), compare-and-swap retry loops (bc's successor bitmap retried $k^2/2$ times per word in a batch) and vector field writes in batches (charged with the whole batch's weight). NPB uses neither the STL nor atomics. The interpreter was frozen at commit `2ee17f3`; the NPB spectra are bit-identical there and at `df6e536`, where they were computed (checked for every MG class and for IS and CG at class S). The runtime baseline was fitted on the 12 serial GAP workloads. The predictions and the protocol were committed (`27be534`) before any NPB trace existed (`paper_generalization/blind2/`).

Results. (Table 19.) IS and CG are predicted almost exactly: the footprint within 0.33% at every class and α within 1.8–5.3%, as good as the best PolyBench kernels, for programs whose control flow depends on random numbers they generate (IS's keys, CG's sparsity pattern), interpreted exactly. MG fails: the footprint is 7–9% short and α is off by 221–297%. MG's class A has a prediction but no truth: Pin's splitter grew to the node's 62 GB and was stopped.

Table 19: The second blind test: committed predictions against Pin. "Program part" is the interpreter's spectrum; the rest is the runtime baseline.

Program	Class	Pin footprint	Predicted	Program part	Footprint error (%)	α MAPE (%)
IS	S	1.00 MB	1.00 MB	0.54 MB	+0.32	4.1
IS	W	9.12 MB	9.12 MB	8.66 MB	+0.03	1.8
CG	S	1.74 MB	1.74 MB	1.28 MB	+0.10	5.3
CG	W	8.38 MB	8.39 MB	7.92 MB	+0.03	4.9
MG	S	1.52 MB	1.41 MB	0.95 MB	-7.26	221.0
MG	W	58.53 MB	53.20 MB	52.74 MB	-9.10	296.5
MG	A	–	412.65 MB	412.19 MB	full trace out of memory	

Post hoc (MG). Looked at after the traces existed: the interpreter's reference count is within ±40% of Pin's (0.77× at S, 1.40× at W, as for IS and CG), yet the predicted bin footprints are 3.5–4.5× smaller than measured at every interval, and the program's footprint is about 10% short of the truth minus the baseline. Since the splitter samples each reference independently (Section 3.2), bins this much smaller with a similar total mean that the interpreter concentrates MG's references on far fewer addresses than the program does. We have not identified the cause (MG indexes its grids through casts to pointers to variable-length arrays, a construct no other workload uses), and report MG as a failure of the method.

Cost. (Table 20.) MG is cheap to interpret (10 s at S, 68 s at W, 409 s at A; 0.27–1.75 of a full trace). IS and CG are not: their inputs come from NPB's linear congruential generator, a chain of `randlc` calls each of which needs the previous one's result, so the interpreter runs them one at a time (about 3 ms each). IS took 17 minutes at S and 4.0 hours at W (185 and 214 full traces), CG 48 minutes and 5.5 hours (47 full traces each). For such programs, a full trace is far cheaper than the prediction.

12 The cost of every method

Measurement. Every cost in this paper was measured afresh for this version, on one 32-core node (62 GB) that was shared with other jobs (median 1-minute load 8.97; each step logs its load, `data/costs/load.txt`). Native and Pin runs used `scripts/run.sh`, one run at a time: three native runs (mean) and one Pin run per mode and config, for the 27 PolyBench kernels at all seven configs, the 12 serial GAP workloads at scales 14, 16 and 18 (full traces at 16), the two C programs, HPCCG, LULESH and NPB (`paper_generalization/costs_pin.sh`, `costs_pin_train.sh`). Everything outside Pin (parse, interpreter, skeleton, analysis) was timed in-process, one step at a time, as the median of three runs when a run takes under 5 s (`costs.py`). The C++ interpreter's long runs (GAP at every scale, HPCCG, LULESH, NPB) are the times recorded when their spectra were computed, often with several runs at a time; GAP's scale-16 runs were repeated alone as a check. `costs_tables.py` builds the tables.

Table 20: Cost of every method on every program set: the native run, Pin's overhead (instrumentation only, the sampled run that MemPrint deploys, the full trace that gives the exact footprint), and each static method's time, as multiples of the native run and of a full trace. PolyBench and GAP rows are medians over their programs; the others are single programs at their largest config.

Programs (size)	native	Pin (×native)			Static method			
	s	instr.	sampled	full trace		s	×native	×trace
PolyBench (27 kernels, MEDIUM)	65 ms	41	44	582	C interpreter, symbolic	174 ms	2.66	0.005
					C interpreter, enumerated	1.9	25	0.042
					C interpreter, timeline	4.7	62	0.11
GAP, 12 workloads (scale 16)	302 ms	14	17	266	hand skeleton	15.0	70	0.21
					C++ interpreter, alone	58.0	176	0.68
					C++ interpreter, 4–16 at a time	68.9	291	1.18
GAP, 12 workloads (scale 18)	1.6	5.56	7.89	–	hand skeleton	66.9	46	–
					C++ interpreter, 4–16 at a time	257	183	–
csr_pr (scale 16)	111 ms	27	32	807	C interpreter	36.0	323	0.40
csr_bfs (scale 14)	18 ms	96	101	393	C interpreter	190	10260	26
HPCCG (28)	109 ms	51	68	1639	C++ interpreter	48.4	445	0.27
LULESH (20)	222 ms	30	39	1001	C++ interpreter	306	1376	1.37
NPB IS (S)	15 ms	158	164	352	C++ interpreter	1007	65095	185
NPB IS (W)	199 ms	17	20	335	C++ interpreter	4.0 h	71546	214
NPB CG (S)	54 ms	61	76	1138	C++ interpreter	2905	53860	47
NPB CG (W)	342 ms	21	38	1228	C++ interpreter	5.5 h	57537	47
NPB MG (S)	13 ms	265	245	451	C++ interpreter	10.1	792	1.75
NPB MG (W)	303 ms	19	28	818	C++ interpreter	67.6	224	0.27
NPB MG (A)	2.5	12	23	out of memory	C++ interpreter	409	163	–

What it shows.

- **Pin.** Instrumentation alone costs 12–265× the native run, the sampled run barely more (8–245×; the ratios are largest for the shortest runs, which Pin's start-up dominates), and a full trace 266–1,639×. Every method that applies a regression model pays at least the sampled run.
- **The static footprint on affine code** costs 0.17 s at MEDIUM with symbolic counting (2.7× native, 0.5% of a full trace, a twentieth of a sampled run) and is the only method cheaper than MemPrint's own deployment. Enumerating every iteration costs 7.79× more and timeline mode 23× more (Section 7.4).
- **The hand skeleton** runs in a fifth of a full trace on GAP (15 s at scale 16, 67 s at 18; median 3.19 s over all scales, at most 118 s), but it is a program written by a person for each kernel: about 1,075 lines of Python for GAP's seven, against about 4,900 lines for the two interpreters, which contain no code for any particular program.

- **The C++ interpreter on irregular code** costs 0.3–2.2 full traces (GAP 0.68 at scale 16 alone, HPCCG 0.27, LULESH 1.4, MG 0.27–1.75), that is 7–35 sampled runs. Where the program is a long chain of dependent steps it must run one at a time and costs far more: NPB IS 185–214 full traces, CG 47, the C program csr_bfs 26.
- **The analysis** that turns spectra, traces and sampled bins into predictions takes at most 0.4 s (Table 21): fitting the runtime baseline by NNLS (388 ms), the pooled regression (253 ms), the known programs' measured descriptors (0.3 s); the moments at 13 intervals take 0.88 ms.
- **One-time costs.** Borrowing, the pooled regression and the runtime baseline need full traces of known programs once: 3,707 s for PolyBench's 27 kernels at seven configs; for GAP, the 12 workloads at nine scales (a median of 90 s each at scale 16 alone).
- **Memory.** A full trace of MG at class A (412 MB footprint) exceeded the node's 62 GB; the C++ interpreter reached 9.5 GB on GAP at scale 18, which limited it to 4–6 runs at a time.

Table 21: The analysis steps on PolyBench, per held-out kernel and split (milliseconds), and the parse-based descriptors at SMALL2 and MEDIUM.

Step	median (ms)	max (ms)	runs
runtime baseline (NNLS on 26)	388.0	409.2	27
moments at 13 intervals (static α)	0.9	1.7	54
static α for the sampled bins	1.0	1.9	54
descriptor $\hat{z}$	1.7	2.4	54
known programs' measured z	305.6	335.1	54
distances and RBF weights	66.7	77.5	54
apply the 26 borrowed models	0.6	0.7	54
nearest model (pick, apply)	0.1	0.2	54
RBF mixture (combine)	0.2	0.3	54
pooled regression with static α (fit and apply)	253.2	270.5	54
own model: fit on 6 configs and apply	6.3	6.9	54
fit the 27 per-program models	115.7	115.7	1
AST node counts	5.3	6.0	54
access-idiom features	57.2	68.8	54
clang parse	33.8	39.6	54

Verification. Table 22 compares the re-measured PolyBench times with those behind the first version of Table 8, taken on the same node (full traces and native runs from the original traces, sampled and instrumentation-only runs four kernels at a time). Medians per kernel agree within 15% (full traces are now 13–15% faster, native runs 4–9%), and the rank correlations over the kernels are 0.89–0.99, except instrumentation only at SMALL2 (0.58, times of 1.4–2.4 s) and the parse (a few milliseconds, now 14–19% slower); the interpreter enumerating every iteration agrees within 6%. The C++ interpreter run alone at GAP scale 16 takes half the time recorded with up to 16 runs at a time.

Table 22: Re-measured times against the times used before (medians over the PolyBench kernels or GAP workloads, the median and range of the per-program ratio, and the Spearman correlation).

Measured	Config	Before (s)	Now (s)	Now/before	range	Spearman
clang parse	MEDIUM	28 ms	33 ms	1.19	1.06–1.41	0.21
clang parse	SMALL2	30 ms	34 ms	1.14	0.98–1.31	-0.30
Instrumentation only	MEDIUM	2.6	2.8	1.07	0.83–1.19	0.98
Instrumentation only	SMALL2	1.7	1.9	1.08	0.88–1.39	0.58
C interpreter (before: enumerated; now: symbolic)	MEDIUM	1.9	174 ms	0.13	0.03–2.28	0.85
C interpreter (before: enumerated; now: symbolic)	SMALL2	293 ms	101 ms	0.42	0.12–2.76	0.85
Sampled run	MEDIUM	3.6	3.3	1.00	0.64–1.15	0.96
Sampled run	SMALL2	1.8	1.9	1.05	0.83–1.16	0.89
Full trace	MEDIUM	60.9	45.1	0.87	0.54–1.19	0.99
Native run	MEDIUM	71 ms	64 ms	0.91	0.48–1.09	0.98
Full trace	SMALL2	7.5	6.7	0.85	0.55–0.99	0.99
Native run	SMALL2	13 ms	12 ms	0.96	0.51–1.18	0.99
C interpreter, enumerated (same method)	MEDIUM	1.9	1.9	0.94	0.48–2.27	0.99
C interpreter, enumerated (same method)	SMALL2	293 ms	305 ms	1.01	0.69–2.80	0.99
C++ interpreter, GAP scale 16 (recorded in parallel, now alone)	16	68.9	58.0	0.50	0.35–1.54	0.78

13 Discussion

How code translates into z. The chain is: code → (abstract execution) → spectrum → ((3)) → bin moments → z. Only the first arrow needs the program; the rest is arithmetic. Which static features matter is therefore not a modelling choice: they are whatever determines how many references each byte receives. For affine code this is the loop bounds and subscripts; for generated irregular inputs, the generator's distribution and the kernel's control flow on it. Syntactic summaries (AST node counts) predict transfer poorly because they do not determine the spectrum.

Predict rather than borrow. Once the spectrum is available, borrowing a model is unnecessary: the static α beats every borrowed model, including the oracle. Similarity in $\hat{z}$ remains useful where the static prediction is not trusted but some program's model must be chosen, and as an out-of-distribution signal (miniVite).

Does the static footprint make MemPrint unnecessary? Where the interpreter applies, it does: on PolyBench the static footprint is more accurate than MemPrint's own model (0.09% against 18.04%), costs less than MemPrint's single sampled run, needs no traces of the program, and makes even the static α redundant (Section 7.7). It does not replace sampling in general:

- It needs the program's whole input in its source or generated from a known distribution, and code the interpreter models. Programs that read data files, use MPI or unmodelled libraries, or whose control depends on data the interpreter cannot compute, fall to the gate; miniVite runs with coverage 0.06.
- It predicts, it does not measure. When the counts are wrong there is no signal: LULESH's footprint was within 4.2%, but its α was 28–43% off at the largest meshes (Section 10), and NPB MG's footprint was 7–9% short with α off by more than 200% at full coverage (Section 11). A sampled run observes the execution that actually happens.
- It needs traces of other programs with the same runtime for the baseline (Section 7.5), and its cost depends on the program's structure (Section 12): with symbolic counting, affine code costs 0.5% of a full trace and about native time even at EXTRALARGE; irregular code costs 0.3–2.2 full traces; and a program that is a long chain of dependent steps (NPB IS and CG, whose inputs come from a sequential random-number generator) costs 47–214 full traces, where tracing is the cheaper way to the exact footprint.
- The footprint over time comes from source too, with the peak within 3.5% on PolyBench (Section 7.4), but at 23 times the cost of the whole-run footprint and only where the interpreter applies; elsewhere it needs the address-sampling and windowed methods of the companion report [24].

So the static footprint is the first choice for programs the gate admits, and MemPrint (or training-free sampling) for the rest.

What generalises. Generic and reusable: the moment formulas, the per-unit counting, the glibc model, the library cost curves (sort, Mersenne Twister, `rand`), the interpreters' batch execution, the `-O3` register and load model and the container models. Specific to a program: only the stub table of command-line values. The hand skeletons were specific to GAP; the C++ interpreter replaces them.

Threats to validity.

- The uniform-graph pr and bfs skeleton results were tuned on their own traces; the Kronecker, new-kernel and thread tests were blind. The interpreter results on GAP were compared with Pin while the interpreter was being developed (scales 10–12 for all kernels); its pr and bfs results at scales 13–18 were not used during development. HPCCG and LULESH (Section 10) are the only fully blind test of the interpreter, and two programs are a small sample.
- The `rand()` cost and the floyd-warshall/nussinov sequential execution were added after their errors were seen.
- One runtime per suite: the baseline is shared because the programs share a compiler, C library and harness. A program with a different runtime needs its own baseline or the gate.
- The splitter's subsamples come from one execution per input; run-to-run variation of the program is not in the bins.

Limitations.

- No symbolic counting. Counting with Polly [13] and Barvinok [33] would make affine spectra independent of input size; the interpreter enumerates instead, in time proportional to the references charged.
- Inputs read from files are outside the approach; only inputs generated from a known distribution are.
- Floating-point values are tracked in scalars and memory but not across every batching path (a nested batch that fills a local array), and values computed differently from the compiled code (vectorised sums, fused multiply-add) can take a different branch near a tie. Converging PageRank (prc) is left out of the interpreter route.
- Code that the interpreter does not model (other library containers, I/O-driven control, MPI) falls to the gate.

14 Related work

Sampling memory references. Trace sampling for cache simulation [14] and statistical cache models [3, 10] estimate miss ratios from sampled reuse; SHARDS [34] samples by hashed address to build miss-ratio curves. Phase-based sampling [30, 35] reduces simulation cost by selecting representative intervals. MemPrint [25] samples references for the footprint and learns the correction per program; this report removes the per-program training.

Locality from program structure. Reuse-distance analysis [22, 4, 9] characterises locality, and Ding and Zhong predict whole-program locality across inputs from a few training runs. The working set model [8] relates footprint to time windows. Polyhedral tools count the integer points of iteration domains and access relations symbolically [13, 33], which yields exact access counts for affine code; our interpreter computes the same quantity by enumeration and extends it to irregular code with generated inputs.

Instrumentation. Pin [20], Valgrind [27] and its heap profilers [29, 26] measure memory exactly at high cost; MemGaze [15] samples traces with hardware support.

15 Conclusion

For a program that has never been traced, the footprint and MemPrint's correction factor α can be predicted from its source code: under Bernoulli sampling both follow in closed form from the access-count spectrum, and the spectrum can be computed by executing the source abstractly, with a small shared baseline for the runtime. On PolyBench this beats every alternative that borrows another program's model and even the program's own model. The same spectrum yields a descriptor of memory behaviour whose distances rank model transfer as well as measured behaviour does. For irregular programs with generated inputs, replaying the program on input drawn from the generator's distribution works, first with hand-written skeletons and now with a C++ interpreter that runs the benchmark's own source.

Reproducibility. Everything in this report is regenerated by

```
python paper_generalization/compute.py                    # data/*.csv
python paper_generalization/overhead.py                   # data/overhead_*.csv
python paper_generalization/baseline_sensitivity.py       # data/baseline_sensitivity*.csv
python paper_generalization/make_figures.py               # figures/*.pdf
python paper_generalization/make_tables.py                # tables/*.tex
```

from the traces and spectra in `data/` (commands for producing those are in the repository's `FINDINGS.md`).

References

[1] Scott Beamer, Krste Asanović, and David Patterson. Direction-optimizing breadth-first search. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2012.

[2] Scott Beamer, Krste Asanović, and David Patterson. The GAP benchmark suite. *arXiv preprint arXiv:1508.03619*, 2015.

[3] Erik Berg and Erik Hagersten. Fast data-locality profiling of native execution. In *Proceedings of the 2005 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, SIGMETRICS '05, page 169–180, New York, NY, USA, 2005. Association for Computing Machinery.

[4] Kristof Beyls and Erik H. D'Hollander. Reuse distance-based cache hint selection. In *Proceedings of the 8th International Euro-Par Conference on Parallel Processing*, Euro-Par '02, page 265–274, Berlin, Heidelberg, 2002. Springer-Verlag.

[5] Ulrik Brandes. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177, 2001.

[6] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. R-MAT: A recursive model for graph mining. In *Proceedings of the SIAM International Conference on Data Mining (SDM)*, pages 442–446, 2004.

[7] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

[8] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[9] Chen Ding and Yutao Zhong. Predicting whole-program locality through reuse distance analysis. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 245–257, 2003.

[10] David Eklov and Erik Hagersten. StatStack: Efficient modeling of LRU caches. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 55–65, 2010.

[11] Free Software Foundation. *The GNU C Library Reference Manual*, 2024. Section "Memory Allocation Tunables" (`M_MMAP_THRESHOLD`), `https://www.gnu.org/software/libc/manual/`.

[12] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, and Assefaw H. Gebremedhin. miniVite: A graph analytics benchmarking tool for massively parallel systems. In *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems Workshop (PMBS)*, 2018.

[13] Tobias Grosser, Armin Größlinger, and Christian Lengauer. Polly — performing polyhedral optimizations on a low-level intermediate representation. *Parallel Processing Letters*, 22(4), 2012.

[14] R. E. Kessler, Mark D. Hill, and David A. Wood. A comparison of trace-sampling techniques for multi-megabyte caches. *IEEE Transactions on Computers*, 43(6):664–675, 1994.

[15] Ozgur O. Kilic, Nathan R. Tallent, Yasodha Suriyakumar, Chenhao Xie, Andrés Marquez, and Stephane Eranian. Memgaze: Rapid and effective load-level memory trace analysis. In *2022 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 484–495, 2022.

[16] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015.

[17] Charles L. Lawson and Richard J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, 1974.

[18] Jure Leskovec, Deepayan Chakrabarti, Jon Kleinberg, Christos Faloutsos, and Zoubin Ghahramani. Kronecker graphs: An approach to modeling networks. *Journal of Machine Learning Research*, 11:985–1042, 2010.

[19] The LLVM Project. *libclang: C Interface to Clang*. `https://clang.llvm.org/doxygen/group__CINDEX.html`.

[20] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.

[21] Makoto Matsumoto and Takuji Nishimura. Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1):3–30, 1998.

[22] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger. Evaluation techniques for storage hierarchies. *IBM Systems Journal*, 9(2):78–117, 1970.

[23] Ulrich Meyer and Peter Sanders. δ-stepping: A parallelizable shortest path algorithm. *Journal of Algorithms*, 49(1):114–152, 2003.

[24] Nafis Mustakin. Estimating how much memory a program uses over time from partial instrumentation of its memory accesses. Technical report, University of California, Riverside, 2026.

[25] Nafis Mustakin, Joseph Manzano, and Daniel Wong. MemPrint: Constructing program memory footprint estimations using statistical methods from sparsely sampled Pin-based memory traces. Manuscript, 2025.

[26] Nicholas Nethercote and Julian Seward. *DHAT: a dynamic heap analysis tool*. Valgrind user manual, `https://valgrind.org/docs/manual/dh-manual.html`.

[27] Nicholas Nethercote and Julian Seward. Valgrind: a framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '07, page 89–100, New York, NY, USA, 2007. Association for Computing Machinery.

[28] Louis-Noël Pouchet. PolyBench/C 4.2.1: The polyhedral benchmark suite, 2016. `https://sourceforge.net/projects/polybench/`.

[29] Julian Seward and Nicholas Nethercote. *Valgrind Massif: Heap Profiler*, 2015. Available at `http://valgrind.org/docs/manual/ms-manual.html`.

[30] Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. Automatically characterizing large scale program behavior. In *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 45–57, 2002.

[31] Charles Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904.

[32] Michael Sutton, Tal Ben-Nun, and Amnon Barak. Optimizing parallel graph connectivity computation via subgraph sampling. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 12–21, 2018.

[33] Sven Verdoolaege, Rachid Seghir, Kristof Beyls, Vincent Loechner, and Maurice Bruynooghe. Counting integer points in parametric polytopes using Barvinok's rational functions. *Algorithmica*, 48(1):37–66, 2007.

[34] Carl A. Waldspurger, Nohhyun Park, Alexander Garthwaite, and Irfan Ahmad. Efficient MRC construction with SHARDS. In *Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST)*, pages 95–110, 2015.

[35] Roland E. Wunderlich, Thomas F. Wenisch, Babak Falsafi, and James C. Hoe. SMARTS: Accelerating microarchitecture simulation via rigorous statistical sampling. In *Proceedings of the 30th Annual International Symposium on Computer Architecture (ISCA)*, pages 84–95, 2003.

A Per-kernel leave-one-out results

B Commands

```
# PolyBench, traced with bytes touched
scripts/run.sh polybench --mode splitter --footprint bytes --runs 1 --bench <kernel>
python -m memprint --root data/polybench-bytes preprocess
python -m memprint --root data/polybench-bytes static spectra --polybench workloads/src/polybench
python -m memprint --root data/polybench-bytes static idioms --polybench workloads/src/polybench
python -m memprint --root data/polybench-bytes static lowo --ast <ast_features.csv> --transfer miniVite
# GAP
scripts/run.sh gapbs --mode splitter --footprint bytes --runs 1 --configs "10 11 12 13 14 15 16 17 18"
python -m memprint --root data/gap-bytes preprocess
python -m memprint --root data/gap-bytes static gap --borrow data/polybench-bytes/data
# C programs
python -m memprint --root data/csr-bytes static programs --borrow data/polybench-bytes/data
```

Table 23: Per held-out PolyBench kernel: footprint error (fp), static α and own model MAPE (%) for both splits, and, for EXTRA, the known kernel nearest by $\hat{z}$ and the oracle.

Kernel	EXTRA			INTER			Nearest by $\hat{z}$ (EXTRA)	
	fp	static α	own	fp	static α	own	chosen	oracle
2mm	0.09	0.7	8.1	0.25	4.7	7.8	gemm	syr2k
3mm	0.06	1.0	9.1	0.17	4.8	11.6	2mm	syr2k
adi	0.11	0.9	32.8	0.31	3.9	7.5	jacobi-2d	cholesky
atax	0.04	4.2	19.5	0.22	9.0	8.4	mvt	bicg
bicg	0.04	3.3	20.2	0.20	9.0	10.5	mvt	mvt
cholesky	0.19	0.6	17.4	1.67	3.5	10.0	lu	ludcmp
correlation	0.55	1.4	12.6	2.59	5.6	10.9	covariance	2mm
covariance	0.06	1.3	15.6	0.27	5.5	12.5	correlation	gramschmidt
doitgen	0.05	1.8	27.1	0.25	6.3	16.1	symm	syr2k
fdtd-2d	0.05	1.2	16.8	0.25	4.8	14.9	adi	cholesky
floyd-warshall	0.13	12.8	17.9	0.63	15.6	10.3	lu	heat-3d
gemm	0.12	1.0	10.6	0.27	4.7	11.8	2mm	gramschmidt
gemver	0.03	3.2	13.8	0.11	8.3	15.0	bicg	mvt
gesummv	0.04	5.2	9.1	0.21	10.6	10.8	trisolv	trisolv
gramschmidt	0.38	1.1	19.8	2.45	5.3	11.3	gemm	gemm
heat-3d	0.13	0.9	20.9	0.29	3.5	13.1	seidel-2d	seidel-2d
jacobi-2d	0.13	1.0	26.5	0.27	3.8	9.9	adi	ludcmp
lu	0.05	0.6	19.4	0.19	2.7	12.0	cholesky	cholesky
ludcmp	0.02	0.6	19.2	0.13	2.8	12.9	cholesky	cholesky
mvt	0.03	3.5	18.0	0.20	10.1	13.4	bicg	bicg
nussinov	0.05	14.3	25.6	0.28	16.3	10.5	cholesky	syrk
seidel-2d	0.11	0.6	32.9	0.31	3.3	8.8	ludcmp	jacobi-2d
symm	0.12	1.0	16.5	0.28	5.3	12.5	correlation	3mm
syr2k	0.11	1.4	15.7	0.26	7.1	11.3	covariance	3mm
syrk	0.15	1.5	18.8	0.33	6.0	10.3	symm	gramschmidt
trisolv	0.07	5.5	12.9	0.36	9.9	8.5	gesummv	gesummv
trmm	0.23	1.2	20.2	0.41	5.7	10.5	correlation	fdtd-2d